

backend/main.py

```
# ===== 文件: backend/main.py =====
1 import logging
2 import sys
3
4 from fastapi import FastAPI
5 from fastapi.middleware.cors import CORSMiddleware
6
7 from app.auth import router as auth_router
8 from app.datasets import router as datasets_router
9 from app.config import Settings, get_settings, validate_jwt_secret_key
10 from app.database import engine
11 from app.migrations import ensure_database_is_current
12 from app.product import SOFTWARE_NAME, SOFTWARE_VERSION
13 from app.task_executor import task_execution_manager
14 from app.tasks import router as tasks_router
15 from app.storage_cleanup import process_pending_cleanup_jobs
16
17
18 settings = get_settings()
19 logger = logging.getLogger(__name__)
20
21
22 def validate_startup_configuration(current_settings: Settings) -> None:
23     """启动前再次校验安全配置，防止绕过 Settings 校验后继续提供服务。"""
24     try:
25         validate_jwt_secret_key(current_settings.jwt_secret_key, current_settings.app_env)
26     except ValueError as exc:
27         raise RuntimeError(f"启动配置无效: {exc}") from exc
28
29
30 validate_startup_configuration(settings)
31
32 app = FastAPI(title=f"{SOFTWARE_NAME} API {SOFTWARE_VERSION}")
33
34 app.add_middleware(
35     CORSMiddleware,
36     allow_origins=settings.allowed_origins,
37     allow_credentials=True,
38     allow_methods=["*"],
39     allow_headers=["*"],
40 )
41
42 app.include_router(auth_router)
43 app.include_router(datasets_router)
44 app.include_router(tasks_router)
45 @app.on_event("startup")
46 def validate_database_schema():
47     """启动时只读核对迁移版本，数据库结构变更必须在部署步骤中完成。"""
48     ensure_database_is_current(engine)
49     try:
50         process_pending_cleanup_jobs(limit=100)
51     except Exception:
52         logger.exception("启动时处理文件清理队列失败，调度 Worker 将继续重试")
53     # pytest 使用独立内存库和依赖覆盖，不启动后台线程以免触碰本地开发数据库。
54     if "pytest" not in sys.modules:
55         task_execution_manager.start()
56
57
58 @app.on_event("shutdown")
59 def stop_task_executor():
60     if "pytest" not in sys.modules:
61         task_execution_manager.stop()
62
63
64 @app.get("/api/health")
65 def health():
```

backend/main.py

```
66     return {"status": "ok", "service": "soft_web_backend"}
67
# ===== 文件: backend/app/product.py =====
1  """软件身份信息的唯一来源。
2
3  软著材料、API 元数据和通知邮件都引用这里的名称与版本，避免不同模块各自维护品牌文本。
4  """
5
6  SOFTWARE_NAME = "新材料数据分析---拓扑感知多核集成聚类分析系统"
7  SOFTWARE_SHORT_NAME = "新材料数据分析系统"
8  SOFTWARE_VERSION = "V1.0"
9
# ===== 文件: backend/app/config.py =====
1  from functools import lru_cache
2  from pathlib import Path
3  from urllib.parse import quote_plus
4
5  from pydantic import Field, model_validator
6  from pydantic_settings import BaseSettings, SettingsConfigDict
7
8
9  BASE_DIR = Path(__file__).resolve().parents[1]
10 JWT_SECRET_PLACEHOLDER = "replace_with_a_random_secret"
11 MIN_JWT_SECRET_BYTES = 32
12
13
14 def validate_jwt_secret_key(value: str | None, app_env: str) -> str:
15     """校验 JWT 密钥，避免应用在空密钥或弱密钥下运行。"""
16     environment = (app_env or "development").strip().lower()
17     secret = (value or "").strip()
18
19     if not secret:
20         if environment == "production":
21             raise ValueError("生产环境必须配置 JWT_SECRET_KEY")
22         raise ValueError(f"{environment} 环境必须配置 JWT_SECRET_KEY")
23     if secret == JWT_SECRET_PLACEHOLDER:
24         raise ValueError("JWT_SECRET_KEY 仍是配置模板占位值，请替换为随机密钥")
25     if len(secret.encode("utf-8")) < MIN_JWT_SECRET_BYTES:
26         raise ValueError(f"JWT_SECRET_KEY 至少需要 {MIN_JWT_SECRET_BYTES} 字节")
27     return secret
28
29
30 class Settings(BaseSettings):
31     app_env: str = Field("development", validation_alias="APP_ENV")
32     max_dataset_file_size_mb: int = Field(100, ge=1, validation_alias="MAX_DATASET_FILE_SIZE_MB")
33     max_matrix_rows: int = Field(50000, ge=1, validation_alias="MAX_MATRIX_ROWS")
34     max_matrix_columns: int = Field(5000, ge=1, validation_alias="MAX_MATRIX_COLUMNS")
35     max_mat_variables: int = Field(100, ge=1, validation_alias="MAX_MAT_VARIABLES")
36     user_storage_quota_mb: int = Field(2048, ge=1, validation_alias="USER_STORAGE_QUOTA_MB")
37     dataset_parse_timeout_seconds: float = Field(30.0, gt=0, validation_alias="DATASET_PARSE_TIMEOUT_SECONDS")
38     db_host: str = Field("127.0.0.1", validation_alias="DB_HOST")
39     db_port: int = Field(3306, validation_alias="DB_PORT")
40     db_name: str = Field("soft_web", validation_alias="DB_NAME")
41     db_user: str = Field("root", validation_alias="DB_USER")
42     db_password: str = Field("", validation_alias="DB_PASSWORD")
43     db_charset: str = Field("utf8mb4", validation_alias="DB_CHARSET")
44     db_pool_size: int = Field(5, validation_alias="DB_POOL_SIZE")
45     db_max_overflow: int = Field(10, validation_alias="DB_MAX_OVERFLOW")
46     db_pool_recycle: int = Field(1800, validation_alias="DB_POOL_RECYCLE")
47     db_echo_sql: bool = Field(False, validation_alias="DB_ECHO_SQL")
48
49     jwt_secret_key: str | None = Field(None, validation_alias="JWT_SECRET_KEY")
50     jwt_algorithm: str = Field("HS256", validation_alias="JWT_ALGORITHM")
51     access_token_expire_minutes: int = Field(30, validation_alias="ACCESS_TOKEN_EXPIRE_MINUTES")
```

backend/app/config.py

```
52     refresh_token_expire_days: int = Field(7, validation_alias="REFRESH_TOKEN_EXPIRE_DAYS")
53     dataset_storage_dir: Path = Field(
54         BASE_DIR / "storage" / "datasets",
55         validation_alias="DATASET_STORAGE_DIR",
56     )
57     result_storage_dir: Path = Field(
58         BASE_DIR / "storage" / "results",
59         validation_alias="RESULT_STORAGE_DIR",
60     )
61     task_executor_enabled: bool = Field(True, validation_alias="TASK_EXECUTOR_ENABLED")
62     task_poll_interval_seconds: float = Field(1.0, validation_alias="TASK_POLL_INTERVAL_SECONDS")
63     task_max_runtime_seconds: float = Field(3600.0, gt=0, validation_alias="TASK_MAX_RUNTIME_SECONDS")
64     task_heartbeat_interval_seconds: float = Field(10.0, gt=0, validation_alias="TASK_HEARTBEAT_INTERVAL_SECONDS")
65     task_heartbeat_timeout_seconds: float = Field(60.0, gt=0, validation_alias="TASK_HEARTBEAT_TIMEOUT_SECONDS")
66     turnstile_secret_key: str = Field("", validation_alias="TURNSTILE_SECRET_KEY")
67     turnstile_verify_url: str = Field(
68         "https://challenges.cloudflare.com/turnstile/v0/siteverify",
69         validation_alias="TURNSTILE_VERIFY_URL",
70     )
71     turnstile_timeout_seconds: float = Field(5.0, validation_alias="TURNSTILE_TIMEOUT_SECONDS")
72     turnstile_pass_expire_minutes: int = Field(5, validation_alias="TURNSTILE_PASS_EXPIRE_MINUTES")
73     email_verification_required: bool = Field(True, validation_alias="EMAIL_VERIFICATION_REQUIRED")
74     email_code_expire_minutes: int = Field(10, validation_alias="EMAIL_CODE_EXPIRE_MINUTES")
75     email_code_resend_cooldown_seconds: int = Field(60, validation_alias="EMAIL_CODE_RESEND_COOLDOWN_SECONDS")
76     smtp_host: str = Field("smtp.qq.com", validation_alias="SMTP_HOST")
77     smtp_port: int = Field(465, validation_alias="SMTP_PORT")
78     smtp_username: str = Field("", validation_alias="SMTP_USERNAME")
79     smtp_password: str = Field("", validation_alias="SMTP_PASSWORD")
80     smtp_from_email: str = Field("", validation_alias="SMTP_FROM_EMAIL")
81     smtp_from_name: str = Field("新材料数据分析系统", validation_alias="SMTP_FROM_NAME")
82     smtp_use_ssl: bool = Field(True, validation_alias="SMTP_USE_SSL")
83     smtp_timeout_seconds: float = Field(10.0, validation_alias="SMTP_TIMEOUT_SECONDS")
84
85     cors_origins: str = Field(
86         "http://localhost:5173,http://127.0.0.1:5173,http://localhost:5174,http://127.0.0.1:5174",
87         validation_alias="CORS_ORIGINS",
88     )
89
90     model_config = SettingsConfigDict(
91         env_file=str(BASE_DIR / "config" / "database.env"),
92         env_file_encoding="utf-8",
93         extra="ignore",
94     )
95
96     @model_validator(mode="after")
97     def validate_security_settings(self):
98         self.jwt_secret_key = validate_jwt_secret_key(self.jwt_secret_key, self.app_env)
99         return self
100
101     @property
102     def database_url(self) -> str:
103         password = quote_plus(self.db_password)
104         return (
105             f"mysql+pymysql://{self.db_user}:{password}"
106             f"@{self.db_host}:{self.db_port}/{self.db_name}?charset={self.db_charset}"
107         )
108
109     @property
110     def allowed_origins(self) -> list[str]:
111         return [origin.strip() for origin in self.cors_origins.split(",") if origin.strip()]
112
113
114     @lru_cache
115     def get_settings() -> Settings:
116         return Settings()
117
```

backend/app/config.py

```
# ===== 文件: backend/app/database.py =====
1 from collections.abc import Generator
2
3 from sqlalchemy import create_engine, event
4 from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker
5 from sqlalchemy.pool import StaticPool
6
7 from .config import get_settings
8
9
10 class Base(DeclarativeBase):
11     pass
12
13
14 def make_engine(database_url: str | None = None):
15     settings = get_settings()
16     url = database_url or settings.database_url
17     kwargs = {"future": True, "echo": settings.db_echo_sql}
18
19     if url.startswith("sqlite"):
20         kwargs.update(
21             {
22                 "connect_args": {"check_same_thread": False},
23                 "poolclass": StaticPool,
24             },
25         )
26     else:
27         kwargs.update(
28             {
29                 "pool_pre_ping": True,
30                 "pool_size": settings.db_pool_size,
31                 "max_overflow": settings.db_max_overflow,
32                 "pool_recycle": settings.db_pool_recycle,
33             },
34         )
35
36     created_engine = create_engine(url, **kwargs)
37
38     if created_engine.dialect.name == "mysql":
39         @event.listens_for(created_engine, "connect")
40         def _set_mysql_utc(dbapi_connection, connection_record) -> None:
41             """让数据库默认时间函数和直接 SQL 写入都使用 UTC。"""
42             cursor = dbapi_connection.cursor()
43             try:
44                 cursor.execute("SET time_zone = '+00:00'")
45             finally:
46                 cursor.close()
47
48     return created_engine
49
50
51 def make_session_factory(engine):
52     return sessionmaker(bind=engine, autoflush=False, autocommit=False, future=True)
53
54
55 engine = make_engine()
56 SessionLocal = make_session_factory(engine)
57
58
59 def get_session() -> Generator[Session, None, None]:
60     with SessionLocal() as session:
61         yield session
62
63
64 # ===== 文件: backend/app/datasets.py =====
```

```
backend/app/datasets.py
```

```
1 import asyncio
2 import hashlib
3 import io
4 import json
5 import logging
6 import multiprocessing
7 import os
8 import pickle
9 import re
10 import shutil
11 import tempfile
12 import zipfile
13 from dataclasses import dataclass
14 from datetime import datetime, timezone
15 from pathlib import Path
16
17 import numpy as np
18 import scipy.io as sio
19 from fastapi import APIRouter, Depends, File, Form, HTTPException, Query, UploadFile
20 from fastapi.responses import StreamingResponse
21 from sqlalchemy import asc, case, desc, func, select
22 from sqlalchemy.exc import IntegrityError
23 from sqlalchemy.orm import Session
24
25 from .auth import get_current_user
26 from .config import get_settings
27 from .database import get_session
28 from .models import AnalysisTask, Dataset, DatasetQuality, DatasetRevision, OperationLog, User
29 from .storage_cleanup import enqueue_cleanups, process_pending_cleanup_jobs
30 from .time_utils import format_utc_iso, utc_now
31 from .schemas import (
32     DatasetBulkRequest,
33     DatasetCatalogItemResponse,
34     DatasetCatalogPageResponse,
35     DatasetParseResponse,
36     DatasetRenameRequest,
37     DatasetRevisionResponse,
38     MessageResponse,
39 )
40
41 router = APIRouter(prefix="/api/datasets", tags=["datasets"])
42 LABEL_VARIABLE_NAMES = ("y", "label", "labels")
43 SAFE_FILENAME_PATTERN = re.compile(r"^[0-9A-Za-z._-]+")
44 UPLOAD_CHUNK_SIZE = 1024 * 1024
45 logger = logging.getLogger(__name__)
46
47
48 class MatValidationError(ValueError):
49     def __init__(self, message: str, status_code: int = 400):
50         super().__init__(message)
51         self.status_code = status_code
52
53
54 @dataclass(frozen=True)
55 class MatLimits:
56     max_file_size_bytes: int
57     max_matrix_rows: int
58     max_matrix_columns: int
59     max_variables: int
60     timeout_seconds: float
61
62
63 @dataclass
64 class StagedUpload:
65     path: Path
66     filename: str
```

```
backend/app/datasets.py
```

```
67     size: int
68     file_hash: str
69
70     def cleanup(self) -> None:
71         # 临时文件不属于数据库事务；无论解析或提交在哪一步失败，都必须单独清理。
72         try:
73             self.path.unlink(missing_ok=True)
74         except OSError:
75             logger.warning("清理临时数据集文件失败: %s", self.path.name)
76
77
78     def _variable_shape(value) -> list[int]:
79         return [int(dim) for dim in getattr(value, "shape", [])]
80
81
82     def _format_label(value) -> str:
83         if isinstance(value, np.generic):
84             value = value.item()
85         if isinstance(value, bytes):
86             value = value.decode(errors="ignore")
87         if isinstance(value, float) and value.is_integer():
88             return str(int(value))
89         return str(value)
90
91
92     def _flatten_values(value) -> np.ndarray:
93         values = np.asarray(value).squeeze().reshape(-1)
94         if np.issubdtype(values.dtype, np.floating):
95             values = values[~np.isnan(values)]
96         return values
97
98
99     def _find_label_variable(mat: dict):
100         for key in LABEL_VARIABLE_NAMES:
101             if key in mat:
102                 return key, mat[key]
103
104         for name, value in mat.items():
105             if name.startswith("__"):
106                 continue
107             if name.lower() in LABEL_VARIABLE_NAMES:
108                 return name, value
109
110         return None, None
111
112
113     def _build_label_distribution(label_values) -> list[dict]:
114         labels = _flatten_values(label_values)
115         if labels.size == 0:
116             return []
117
118         unique_labels, counts = np.unique(labels, return_counts=True)
119         total = int(counts.sum())
120
121         return [
122             {
123                 "label": _format_label(label),
124                 "count": int(count),
125                 "percent": round(int(count) * 100 / total) if total else 0,
126             }
127             for label, count in zip(unique_labels, counts)
128         ]
129
130
131     def _format_value_range(values: np.ndarray) -> str:
132         if values.size == 0:
```

backend/app/datasets.py

```

133         return ""
134
135     if np.issubdtype(values.dtype, np.number):
136         numeric_values = values.astype(float)
137         if numeric_values.size == 0:
138             return ""
139         return f"{{format_label(float(numeric_values.min()))}} - {{format_label(float(numeric_values.max()))}}"
140
141     unique_values = np.unique(values)
142     if unique_values.size == 1:
143         return _format_label(unique_values[0])
144     return f"{{format_label(unique_values[0])}} - {{format_label(unique_values[-1])}}"
145
146
147 def _build_cluster_stats(matrix) -> list[dict]:
148     values = np.asarray(matrix)
149     if values.ndim < 2:
150         return []
151
152     stats = []
153     for index in range(values.shape[1]):
154         column_values = _flatten_values(values[:, index])
155         stats.append(
156             {
157                 "name": f"base_{index + 1}",
158                 "clusterCount": int(np.unique(column_values).size),
159                 "range": _format_value_range(column_values),
160             },
161         )
162
163     return stats
164
165
166 def _safe_filename(filename: str) -> str:
167     source = Path(filename).name
168     suffix = Path(source).suffix.lower()
169     stem = Path(source).stem or "dataset"
170     safe_stem = SAFE_FILENAME_PATTERN.sub("_", stem).strip("._-") or "dataset"
171     return f"{{safe_stem[:80]}}{{suffix[:16]}}"
172
173
174 def _dataset_name(filename: str) -> str:
175     return (Path(filename).stem or "未命名数据集")[:128]
176
177
178 def _mat_limits() -> MatLimits:
179     settings = get_settings()
180     return MatLimits(
181         max_file_size_bytes=int(settings.max_dataset_file_size_mb) * 1024 * 1024,
182         max_matrix_rows=int(settings.max_matrix_rows),
183         max_matrix_columns=int(settings.max_matrix_columns),
184         max_variables=int(settings.max_mat_variables),
185         timeout_seconds=float(settings.dataset_parse_timeout_seconds),
186     )
187
188
189 def _visible_mat_variables(mat: dict) -> dict:
190     return {name: value for name, value in mat.items() if not name.startswith("__")}
191
192
193 def _validate_loaded_mat(mat: dict, limits: MatLimits) -> None:
194     """在子进程内完成结构校验，避免畸形数组进入主进程和数据库事务。"""
195     variables = _visible_mat_variables(mat)
196     if not variables:
197         raise MatValidationError(".mat 文件不包含可用变量")
198     if len(variables) > limits.max_variables:

```

backend/app/datasets.py

```
199         raise MatValidationError(
200             f".mat 文件变量数量超过限制（最多 {limits.max_variables} 个）",
201             status_code=413,
202         )
203
204     for name, value in variables.items():
205         if not isinstance(value, np.ndarray):
206             raise MatValidationError(f"变量 “{name}” 结构异常，仅支持 NumPy 数组")
207         if value.dtype.hasobject or value.dtype.kind in {"O", "V"}:
208             raise MatValidationError(f"变量 “{name}” 包含不支持的对象类型")
209         if value.ndim not in {1, 2}:
210             raise MatValidationError(f"变量 “{name}” 结构异常，仅支持一维或二维数组")
211         if any(int(dimension) <= 0 for dimension in value.shape):
212             raise MatValidationError(f"变量 “{name}” 包含空维度")
213
214         if value.ndim == 1 and value.shape[0] > limits.max_matrix_rows:
215             raise MatValidationError(
216                 f"变量 “{name}” 长度超过限制（最多 {limits.max_matrix_rows} 行）",
217                 status_code=413,
218             )
219         if value.ndim == 2:
220             rows, columns = (int(value.shape[0]), int(value.shape[1]))
221             if rows > limits.max_matrix_rows:
222                 raise MatValidationError(
223                     f"矩阵行数超过限制（最多 {limits.max_matrix_rows} 行）",
224                     status_code=413,
225                 )
226             if columns > limits.max_matrix_columns:
227                 raise MatValidationError(
228                     f"矩阵列数超过限制（最多 {limits.max_matrix_columns} 列）",
229                     status_code=413,
230                 )
231
232         if np.issubdtype(value.dtype, np.number):
233             try:
234                 finite = np.isfinite(value)
235             except TypeError as exc:
236                 raise MatValidationError(f"变量 “{name}” 数值结构异常") from exc
237             if not bool(np.all(finite)):
238                 raise MatValidationError(f"变量 “{name}” 包含 NaN 或 Inf")
239
240     variable_info = {
241         "shape": _variable_shape(value), "dtype": str(value.dtype)}
242     for name, value in variables.items()
243 }
244 main_variable, _ = _find_main_variable(mat, variable_info)
245 if not main_variable:
246     raise MatValidationError("未检测到有效的二维基础聚类矩阵")
247 main_matrix = variables[main_variable]
248 if main_matrix.ndim != 2 or not np.issubdtype(main_matrix.dtype, np.number):
249     raise MatValidationError("基础聚类矩阵必须是二维数值数组")
250
251
252 def _mat_parse_worker(
253     source_path: str,
254     result_path: str,
255     filename: str,
256     max_matrix_rows: int,
257     max_matrix_columns: int,
258     max_variables: int,
259     result_queue,
260 ) -> None:
261     limits = MatLimits(
262         max_file_size_bytes=0,
263         max_matrix_rows=max_matrix_rows,
264         max_matrix_columns=max_matrix_columns,
```


backend/app/datasets.py

```
265         max_variables=max_variables,
266         timeout_seconds=0,
267     )
268     try:
269         mat = sio.loadmat(source_path)
270         _validate_loaded_mat(mat, limits)
271         parsed = _parse_loaded_mat(filename, mat)
272         if result_path:
273             with Path(result_path).open("wb") as output:
274                 pickle.dump(_visible_mat_variables(mat), output, protocol=pickle.HIGHEST_PROTOCOL)
275             result_queue.put({"ok": True, "parsed": parsed})
276     except MatValidationError as exc:
277         result_queue.put(
278             {
279                 "ok": False,
280                 "statusCode": exc.status_code,
281                 "message": str(exc),
282             },
283         )
284     except Exception:
285         result_queue.put(
286             {
287                 "ok": False,
288                 "statusCode": 400,
289                 "message": "无法解析 .mat 文件",
290             },
291         )
292
293
294 def _controlled_parse_mat_file(
295     source_path: Path,
296     filename: str,
297     *,
298     include_mat: bool = False,
299     limits: MatLimits | None = None,
300 ) -> tuple[dict, dict | None]:
301     """以受控 spawn 子进程解析 MAT 文件，并由父进程负责超时和资源回收。"""
302     limits = limits or _mat_limits()
303     try:
304         file_size = source_path.stat().st_size
305     except OSError as exc:
306         raise HTTPException(400, "上传文件临时副本不可用") from exc
307     if file_size <= 0:
308         raise HTTPException(400, "上传文件为空")
309     if file_size > limits.max_file_size_bytes:
310         raise HTTPException(
311             413,
312             f"文件大小超过限制（最大 {limits.max_file_size_bytes // (1024 * 1024)} MB）",
313         )
314
315     result_path: Path | None = None
316     if include_mat:
317         result_file = tempfile.NamedTemporaryFile(prefix="soft-web-mat-", suffix=".pickle", delete=False)
318         result_path = Path(result_file.name)
319         result_file.close()
320
321     context = multiprocessing.get_context("spawn")
322     result_queue = context.Queue(maxsize=1)
323     process = context.Process(
324         target=_mat_parse_worker,
325         args=(
326             str(source_path),
327             str(result_path) if result_path else "",
328             _safe_filename(filename),
329             limits.max_matrix_rows,
330             limits.max_matrix_columns,
```

```
backend/app/datasets.py
331         limits.max_variables,
332         result_queue,
333     ),
334 )
335
336     try:
337         process.start()
338         process.join(limits.timeout_seconds)
339         if process.is_alive():
340             process.terminate()
341             process.join(2)
342             if process.is_alive():
343                 process.kill()
344                 process.join()
345             raise HTTPException(408, "数据集解析超时, 请检查文件复杂度")
346
347     try:
348         result = result_queue.get(timeout=2)
349     except Exception as exc:
350         raise HTTPException(400, "无法解析 .mat 文件") from exc
351
352     if not result.get("ok"):
353         status_code = int(result.get("statusCode") or 400)
354         message = str(result.get("message") or "无法解析 .mat 文件")
355         raise HTTPException(status_code, message)
356
357     parsed = result["parsed"]
358     parsed_mat = None
359     if result_path is not None:
360         with result_path.open("rb") as source:
361             parsed_mat = pickle.load(source)
362     return parsed, parsed_mat
363 finally:
364     if result_path is not None:
365         try:
366             result_path.unlink(missing_ok=True)
367         except OSError:
368             logger.warning("清理矩阵临时交换文件失败: %s", result_path.name)
369     result_queue.close()
370     result_queue.join_thread()
371     if process.pid is not None:
372         process.close()
373
374
375 def _find_main_variable(mat: dict, variables: dict) -> tuple[str | None, list[int]]:
376     # 默认优先使用 E; 没有 E 时, 使用面积最大的二维矩阵作为基础聚类矩阵。
377     main_var = None
378     main_shape = [0, 0]
379     if "E" in mat and len(_variable_shape(mat["E"])) >= 2:
380         main_var = "E"
381         main_shape = _variable_shape(mat["E"])
382
383     for name, info in variables.items():
384         if main_var == "E":
385             break
386         shape = info["shape"]
387         if len(shape) >= 2 and shape[0] * shape[1] > main_shape[0] * main_shape[1]:
388             main_var = name
389             main_shape = shape
390
391     return main_var, main_shape
392
393
394 def _parse_loaded_mat(filename: str, mat: dict) -> dict:
395     variables = {}
396     for name, value in _visible_mat_variables(mat).items():
```

backend/app/datasets.py

```
397     variables[name] = {
398         "shape": _variable_shape(value),
399         "dtype": str(value.dtype),
400     }
401
402     main_var, main_shape = _find_main_variable(mat, variables)
403     label_var, label_values = _find_label_variable(mat)
404     label_distribution = _build_label_distribution(label_values) if label_values is not None else []
405     has_labels = len(label_distribution) > 0
406     sample_count = int(main_shape[0]) if len(main_shape) >= 1 else 0
407     base_count = int(main_shape[1]) if len(main_shape) >= 2 else 0
408     main_matrix = mat[main_var] if main_var else None
409
410     return {
411         "fileName": filename,
412         "variables": variables,
413         "mainVariable": main_var,
414         "labelVariable": label_var,
415         "sampleCount": sample_count,
416         "baseCount": base_count,
417         "classCount": len(label_distribution),
418         "hasLabels": has_labels,
419         "matrixShape": f"{main_var}: {sample_count} x {base_count}" if main_var else "",
420         "labelShape": f"{label_var}: {sample_count}" if label_var else "",
421         "labelDistribution": label_distribution,
422         "clusterStats": _build_cluster_stats(main_matrix) if main_matrix is not None else [],
423     }
424
425
426 def _parse_mat_content(filename: str, content: bytes) -> dict:
427     if not filename.lower().endswith(".mat"):
428         raise HTTPException(400, f"不支持的文件格式: {filename}, 目前仅支持 .mat")
429     limits = _mat_limits()
430     if len(content) > limits.max_file_size_bytes:
431         raise HTTPException(
432             413,
433             f"文件大小超过限制 (最大 {limits.max_file_size_bytes // (1024 * 1024)} MB)",
434         )
435     staged_file = tempfile.NamedTemporaryFile(prefix="soft-web-mat-", suffix=".mat", delete=False)
436     staged_path = Path(staged_file.name)
437     try:
438         staged_file.write(content)
439         staged_file.close()
440         parsed, _ = _controlled_parse_mat_file(staged_path, filename, limits=limits)
441         return parsed
442     finally:
443         try:
444             staged_file.close()
445         except OSError:
446             pass
447         try:
448             staged_path.unlink(missing_ok=True)
449         except OSError:
450             logger.warning("清理矩阵临时文件失败: %s", staged_path.name)
451
452
453 def _persist_parse_log(
454     session: Session,
455     user: User,
456     *,
457     filename: str,
458     success: bool,
459     detail: dict,
460 ) -> None:
461     """解析接口单独提交审计日志, 避免日志事务影响文件解析结果。"""
462     session.add(
```

backend/app/datasets.py

```
463         OperationLog(
464             user_id=user.id,
465             action="dataset_parse" if success else "dataset_parse_failed",
466             level="info" if success else "error",
467             message=("数据集文件解析成功: " if success else "数据集文件解析失败: ") + filename,
468             detail_json=json.dumps(detail, ensure_ascii=False),
469         ),
470     )
471     try:
472         session.commit()
473     except Exception:
474         session.rollback()
475         logger.exception("写入数据集解析操作日志失败")
476
477
478 def _build_quality_summary(parsed: dict) -> tuple[str, list[str]]:
479     issues: list[str] = []
480     sample_count = int(parsed.get("sampleCount") or 0)
481     base_count = int(parsed.get("baseCount") or 0)
482     main_variable = parsed.get("mainVariable")
483
484     if not main_variable or sample_count <= 0 or base_count <= 0:
485         return "error", ["未检测到有效的基础聚类矩阵"]
486
487     if not parsed.get("hasLabels"):
488         issues.append("未提供真实标签, 部分评估指标不可用")
489
490     label_shape = parsed.get("variables", {}).get(parsed.get("labelVariable"), {}).get("shape", [])
491     if label_shape:
492         label_count = int(np.prod(label_shape))
493         if label_count != sample_count:
494             issues.append("标签数量与样本数量不一致")
495
496     return ("warning" if issues else "ready"), issues
497
498
499 def _decode_issues(value: str | None) -> list[str]:
500     if not value:
501         return []
502     try:
503         decoded = json.loads(value)
504     except json.JSONDecodeError:
505         return []
506     return [str(item) for item in decoded] if isinstance(decoded, list) else []
507
508
509 def _upsert_quality(session: Session, dataset: Dataset, parsed: dict) -> DatasetQuality:
510     status, issues = _build_quality_summary(parsed)
511     quality = session.scalar(select(DatasetQuality).where(DatasetQuality.dataset_id == dataset.id))
512     if quality is None:
513         quality = DatasetQuality(dataset_id=dataset.id, status=status, issues_json=json.dumps(issues,
514             ensure_ascii=False))
515         session.add(quality)
516     else:
517         quality.status = status
518         quality.issues_json = json.dumps(issues, ensure_ascii=False)
519     return quality
520
521
522 def _quality_for_dataset(session: Session, dataset: Dataset, parsed: dict | None = None) -> DatasetQuality:
523     quality = session.scalar(select(DatasetQuality).where(DatasetQuality.dataset_id == dataset.id))
524     if quality is not None:
525         return quality
526
527     parsed = parsed or _parse_stored_dataset_file(dataset)
528     quality = _upsert_quality(session, dataset, parsed)
```

backend/app/datasets.py

```
528     session.flush()
529     return quality
530
531
532 def _next_version(session: Session, dataset_id: int) -> int:
533     current = session.scalar(
534         select(func.max(DatasetRevision.version)).where(DatasetRevision.dataset_id == dataset_id),
535     )
536     return int(current or 0) + 1
537
538
539 def _record_revision(
540     session: Session,
541     dataset: Dataset,
542     parsed: dict,
543     action: str,
544     quality: DatasetQuality | None = None,
545 ) -> DatasetRevision:
546     quality = quality or _quality_for_dataset(session, dataset, parsed)
547     revision = DatasetRevision(
548         dataset_id=dataset.id,
549         version=_next_version(session, dataset.id),
550         action=action,
551         name=dataset.name,
552         original_filename=dataset.original_filename,
553         storage_path=dataset.storage_path,
554         file_hash=dataset.file_hash,
555         sample_count=dataset.sample_count,
556         base_cluster_count=dataset.base_cluster_count,
557         has_ground_truth=dataset.has_ground_truth,
558         cluster_count=dataset.cluster_count,
559         quality_status=quality.status,
560         quality_issues_json=quality.issues_json,
561     )
562     session.add(revision)
563     return revision
564
565
566 def _parse_stored_dataset_file(dataset: Dataset) -> dict:
567     path = Path(dataset.storage_path)
568     if not path.exists():
569         return {}
570
571     try:
572         parsed, _ = _controlled_parse_mat_file(path, dataset.original_filename)
573         return parsed
574     except HTTPException:
575         return {}
576
577
578 def _stored_file_size(dataset: Dataset) -> int:
579     path = Path(dataset.storage_path)
580     if not path.exists():
581         return 0
582     return path.stat().st_size
583
584
585 def _task_summaries(
586     session: Session,
587     user_id: int,
588     dataset_ids: list[int],
589 ) -> dict[int, tuple[int, datetime | None]]:
590     if not dataset_ids:
591         return {}
592
593     rows = session.execute(
```

```

backend/app/datasets.py
594         select(
595             AnalysisTask.dataset_id,
596             func.count(AnalysisTask.id),
597             func.max(
598                 case(
599                     (AnalysisTask.status == "succeeded", AnalysisTask.finished_at),
600                     else_=None,
601                 ),
602             ),
603         )
604         .where(
605             AnalysisTask.user_id == user_id,
606             AnalysisTask.dataset_id.in_(dataset_ids),
607         )
608         .group_by(AnalysisTask.dataset_id),
609     ).all()
610     return {int(dataset_id): (int(count), last_finished_at) for dataset_id, count, last_finished_at in rows}
611
612
613 def _catalog_item(
614     session: Session,
615     dataset: Dataset,
616     parsed: dict | None = None,
617     quality: DatasetQuality | None = None,
618     task_summary: tuple[int, datetime | None] | None = None,
619 ) -> DatasetCatalogItemResponse:
620     parsed = parsed or _parse_stored_dataset_file(dataset)
621     sample_count = int(parsed.get("sampleCount") or dataset.sample_count)
622     base_count = int(parsed.get("baseCount") or dataset.base_cluster_count)
623     has_labels = bool(parsed.get("hasLabels", dataset.has_ground_truth))
624     class_count = int(parsed.get("classCount") or dataset.cluster_count or 0) if has_labels else 0
625     quality = quality or _quality_for_dataset(session, dataset, parsed)
626     current_version = session.scalar(
627         select(func.max(DatasetRevision.version)).where(DatasetRevision.dataset_id == dataset.id),
628     )
629     task_count, last_analysis_at = task_summary or _task_summaries(
630         session,
631         dataset.user_id,
632         [dataset.id],
633     ).get(dataset.id, (0, None))
634
635     return DatasetCatalogItemResponse(
636         id=dataset.id,
637         name=dataset.name,
638         createdAt=format_utc_iso(dataset.created_at) or "",
639         fileSizeBytes=_stored_file_size(dataset),
640         sampleCount=sample_count,
641         baseCount=base_count,
642         classCount=class_count,
643         hasLabels=has_labels,
644         taskCount=task_count,
645         lastAnalysisAt=format_utc_iso(last_analysis_at),
646         version=int(current_version or 1),
647         qualityStatus=quality.status,
648         qualityIssues=_decode_issues(quality.issues_json),
649         matrixShape=parsed.get("matrixShape") or f"E: {sample_count} x {base_count}",
650         labelShape=parsed.get("labelShape") or (f"y: {sample_count}" if has_labels else ""),
651         labelDistribution=parsed.get("labelDistribution") or [],
652         clusterStats=parsed.get("clusterStats") or [],
653     )
654
655
656 async def _stage_upload(file: UploadFile) -> StagedUpload:
657     """先分块写入临时文件并校验大小，解析和正式落盘阶段不再持有完整文件字节。"""
658     filename = file.filename or ""
659     if not filename:

```

backend/app/datasets.py

```
660         raise HTTPException(400, "未选择文件")
661     if Path(filename).suffix.lower() != ".mat":
662         raise HTTPException(400, f"不支持的文件格式: {filename}, 目前仅支持 .mat")
663
664     limits = _mat_limits()
665     staged_file = tempfile.NamedTemporaryFile(prefix="soft-web-upload-", suffix=".mat", delete=False)
666     staged_path = Path(staged_file.name)
667     digest = hashlib.sha256()
668     size = 0
669     try:
670         while True:
671             chunk = await file.read(UPLOAD_CHUNK_SIZE)
672             if not chunk:
673                 break
674             size += len(chunk)
675             if size > limits.max_file_size_bytes:
676                 raise HTTPException(
677                     413,
678                     f"文件大小超过限制 (最大 {limits.max_file_size_bytes // (1024 * 1024)} MB)",
679                 )
680             staged_file.write(chunk)
681             digest.update(chunk)
682     staged_file.close()
683     if size == 0:
684         raise HTTPException(400, "上传文件为空")
685     return StagedUpload(
686         path=staged_path,
687         filename=filename,
688         size=size,
689         file_hash=digest.hexdigest(),
690     )
691 except Exception:
692     try:
693         staged_file.close()
694     except OSError:
695         pass
696     try:
697         staged_path.unlink(missing_ok=True)
698     except OSError:
699         logger.warning("清理上传临时文件失败: %s", staged_path.name)
700     raise
701
702
703 def _storage_directory(user_id: int) -> Path:
704     settings = get_settings()
705     return Path(settings.dataset_storage_dir) / str(user_id)
706
707
708 def _storage_usage_bytes(user_id: int) -> int:
709     storage_dir = _storage_directory(user_id)
710     if not storage_dir.exists():
711         return 0
712     total = 0
713     for path in storage_dir.rglob("*"):
714         try:
715             if path.is_file():
716                 total += path.stat().st_size
717         except OSError:
718             continue
719     return total
720
721
722 def _ensure_user_storage_quota(user_id: int, additional_bytes: int) -> None:
723     settings = get_settings()
724     quota_bytes = int(settings.user_storage_quota_mb) * 1024 * 1024
725     used_bytes = _storage_usage_bytes(user_id)
```

backend/app/datasets.py

```
726     if used_bytes + additional_bytes > quota_bytes:
727         used_mb = round(used_bytes / (1024 * 1024), 2)
728         quota_mb = int(settings.user_storage_quota_mb)
729         raise HTTPException(
730             413,
731             f"用户存储空间不足: 已使用 {used_mb} MB, 配额为 {quota_mb} MB",
732         )
733
734
735 def _file_sha256(path: Path) -> str:
736     digest = hashlib.sha256()
737     with path.open("rb") as source:
738         while True:
739             chunk = source.read(UPLOAD_CHUNK_SIZE)
740             if not chunk:
741                 break
742             digest.update(chunk)
743     return digest.hexdigest()
744
745
746 def _save_staged_upload(user_id: int, staged: StagedUpload, filename: str | None = None) -> tuple[str, Path]:
747     """通过同目录临时文件完成原子落盘, 避免数据库记录指向未写完的文件。"""
748     filename = filename or staged.filename
749     storage_dir = _storage_directory(user_id)
750     storage_dir.mkdir(parents=True, exist_ok=True)
751
752     timestamp = datetime.now(timezone.utc).strftime("%Y%m%d%H%M%S%f")
753     storage_path = storage_dir / f"{timestamp}_{staged.file_hash[:12]}_{_safe_filename(filename)}"
754     temporary_storage = tempfile.NamedTemporaryFile(
755         prefix="soft-web-storage-",
756         suffix=".tmp",
757         dir=storage_dir,
758         delete=False,
759     )
760     temporary_path = Path(temporary_storage.name)
761     try:
762         temporary_storage.close()
763         with staged.path.open("rb") as source, temporary_path.open("wb") as destination:
764             shutil.copyfileobj(source, destination, length=UPLOAD_CHUNK_SIZE)
765         os.replace(temporary_path, storage_path)
766     except Exception:
767         try:
768             temporary_path.unlink(missing_ok=True)
769         except OSError:
770             logger.warning("清理存储临时文件失败: %s", temporary_path.name)
771         raise
772
773     return staged.file_hash, storage_path
774
775
776 def _raw_label_values(value) -> np.ndarray:
777     return np.asarray(value).squeeze().reshape(-1)
778
779
780 def _merge_label_values(current_value, incoming_value) -> np.ndarray:
781     current_labels = _raw_label_values(current_value)
782     incoming_labels = _raw_label_values(incoming_value)
783     combined_labels = np.concatenate([current_labels, incoming_labels], axis=0)
784
785     current_array = np.asarray(current_value)
786     if current_array.ndim == 2 and current_array.shape[1] == 1:
787         return combined_labels.reshape(-1, 1)
788     if current_array.ndim == 2 and current_array.shape[0] == 1:
789         return combined_labels.reshape(1, -1)
790     return combined_labels
791
```


backend/app/datasets.py

```
792
793 def _build_appended_mat_file(
794     current_path: Path,
795     current_filename: str,
796     incoming_path: Path,
797     incoming_filename: str,
798     output_path: Path,
799     limits: MatLimits,
800 ) -> dict:
801     """在独立输出文件中合并矩阵；原版本只有在数据库提交成功后才进入清理队列。"""
802     current_parsed, current_mat = _controlled_parse_mat_file(
803         current_path,
804         current_filename,
805         include_mat=True,
806         limits=limits,
807     )
808     incoming_parsed, incoming_mat = _controlled_parse_mat_file(
809         incoming_path,
810         incoming_filename,
811         include_mat=True,
812         limits=limits,
813     )
814     if current_mat is None or incoming_mat is None:
815         raise HTTPException(400, "无法读取追加数据的矩阵内容")
816
817     current_main_var = current_parsed.get("mainVariable")
818     incoming_main_var = incoming_parsed.get("mainVariable")
819     if not current_main_var:
820         raise HTTPException(400, "当前数据集缺少有效的基础聚类矩阵，无法追加")
821     if not incoming_main_var:
822         raise HTTPException(400, "追加文件缺少有效的基础聚类矩阵")
823
824     current_matrix = np.asarray(current_mat[current_main_var])
825     incoming_matrix = np.asarray(incoming_mat[incoming_main_var])
826     if current_matrix.ndim < 2 or incoming_matrix.ndim < 2:
827         raise HTTPException(400, "基础聚类矩阵必须是二维结构，无法追加")
828     if current_matrix.shape[1:] != incoming_matrix.shape[1:]:
829         raise HTTPException(400, "追加文件的基础聚类矩阵列数必须与当前数据集一致")
830
831     current_has_labels = bool(current_parsed.get("hasLabels"))
832     incoming_has_labels = bool(incoming_parsed.get("hasLabels"))
833     if current_has_labels != incoming_has_labels:
834         raise HTTPException(400, "追加文件的标签状态必须与当前数据集一致")
835
836     combined_mat = _visible_mat_variables(current_mat)
837     combined_mat[current_main_var] = np.concatenate([current_matrix, incoming_matrix], axis=0)
838
839     if current_has_labels:
840         current_label_var = current_parsed.get("labelVariable")
841         incoming_label_var = incoming_parsed.get("labelVariable")
842         if not current_label_var or not incoming_label_var:
843             raise HTTPException(400, "追加文件的标签变量无法识别")
844
845         current_labels = _raw_label_values(current_mat[current_label_var])
846         incoming_labels = _raw_label_values(incoming_mat[incoming_label_var])
847         if current_labels.size != int(current_parsed.get("sampleCount") or 0):
848             raise HTTPException(400, "当前数据集的标签数量与样本数量不一致，无法追加")
849         if incoming_labels.size != int(incoming_parsed.get("sampleCount") or 0):
850             raise HTTPException(400, "追加文件的标签数量与样本数量不一致")
851
852         combined_mat[current_label_var] = _merge_label_values(
853             current_mat[current_label_var],
854             incoming_mat[incoming_label_var],
855         )
856
857     _validate_loaded_mat(combined_mat, limits)
```

backend/app/datasets.py

```
858     sio.savemat(str(output_path), combined_mat)
859     output_size = output_path.stat().st_size
860     if output_size > limits.max_file_size_bytes:
861         raise HTTPException(
862             413,
863             f"追加后的文件大小超过限制（最大 {limits.max_file_size_bytes // (1024 * 1024)} MB）",
864         )
865     parsed, _ = _controlled_parse_mat_file(output_path, current_filename, limits=limits)
866     return parsed
867
868
869 def _get_user_dataset(session: Session, dataset_id: int, user: User) -> Dataset:
870     dataset = session.scalar(
871         select(Dataset).where(Dataset.id == dataset_id, Dataset.user_id == user.id),
872     )
873     if dataset is None:
874         raise HTTPException(404, "数据集不存在")
875     return dataset
876
877
878 def _find_name_conflict(
879     session: Session,
880     user: User,
881     name: str,
882     exclude_dataset_id: int | None = None,
883 ) -> Dataset | None:
884     stmt = select(Dataset).where(
885         Dataset.user_id == user.id,
886         func.lower(Dataset.name) == name.strip().lower(),
887     )
888     if exclude_dataset_id is not None:
889         stmt = stmt.where(Dataset.id != exclude_dataset_id)
890     return session.scalar(stmt)
891
892
893 def _ensure_quality_records(session: Session, datasets: list[Dataset]) -> None:
894     updated = False
895     for dataset in datasets:
896         quality = session.scalar(select(DatasetQuality.id).where(DatasetQuality.dataset_id == dataset.id))
897         if quality is None:
898             _quality_for_dataset(session, dataset)
899             updated = True
900     if updated:
901         session.commit()
902
903
904 def _delete_datasets(session: Session, datasets: list[Dataset]) -> set[tuple[str, Path]]:
905     """删除前检查正式任务引用，并把所有版本文件登记到事务性清理队列。"""
906     dataset_ids = [dataset.id for dataset in datasets]
907     task_count = session.scalar(
908         select(func.count(AnalysisTask.id)).where(AnalysisTask.dataset_id.in_(dataset_ids)),
909     )
910     if task_count:
911         raise HTTPException(409, f"数据集仍被 {task_count} 个分析任务引用，请先删除关联任务后再删除数据集")
912
913     revision_paths = session.scalars(
914         select(DatasetRevision.storage_path).where(DatasetRevision.dataset_id.in_(dataset_ids)),
915     ).all()
916     cleanup_paths = {("dataset", Path(path)) for path in revision_paths}
917     cleanup_paths.update(("dataset", Path(dataset.storage_path)) for dataset in datasets)
918
919     # 文件清理任务和数据库删除必须处于同一事务；事务回滚时文件仍保持可用。
920     enqueue_cleanups(session, cleanup_paths)
921
922     for dataset in datasets:
923         session.delete(dataset)
```

backend/app/datasets.py

```
924     try:
925         session.commit()
926     except IntegrityError as exc:
927         session.rollback()
928         raise HTTPException(409, "数据集已被分析任务引用, 请先删除关联任务后再删除数据集") from exc
929     return cleanup_paths
930
931
932 @router.get("", response_model=DatasetCatalogPageResponse)
933 def list_datasets(
934     q: str = "",
935     data_type: str | None = Query(default=None, alias="dataType"),
936     label_status: str = Query(default="all", alias="labelStatus"),
937     usage: str = "all",
938     quality_status: str = Query(default="all", alias="qualityStatus"),
939     sort_by: str = Query(default="createdAt", alias="sortBy"),
940     sort_order: str = Query(default="desc", alias="sortOrder"),
941     page: int = Query(default=1, ge=1),
942     page_size: int = Query(default=20, ge=1, le=100, alias="pageSize"),
943     user: User = Depends(get_current_user),
944     session: Session = Depends(get_session),
945 ):
946     all_user_datasets = session.scalars(select(Dataset).where(Dataset.user_id == user.id)).all()
947     _ensure_quality_records(session, all_user_datasets)
948
949     stmt = select(Dataset).where(Dataset.user_id == user.id)
950     if q.strip():
951         stmt = stmt.where(Dataset.name.ilike(f"%{q.strip()}%"))
952     if data_type == "混合":
953         stmt = stmt.where(False)
954     if label_status == "labeled":
955         stmt = stmt.where(Dataset.has_ground_truth.is_(True))
956     elif label_status == "unlabeled":
957         stmt = stmt.where(Dataset.has_ground_truth.is_(False))
958     if quality_status in {"ready", "warning", "error"}:
959         stmt = stmt.join(DatasetQuality).where(DatasetQuality.status == quality_status)
960     if usage in {"used", "unused"}:
961         referenced_dataset_ids = select(AnalysisTask.dataset_id).where(AnalysisTask.user_id == user.id)
962         if usage == "used":
963             stmt = stmt.where(Dataset.id.in_(referenced_dataset_ids))
964         else:
965             stmt = stmt.where(Dataset.id.not_in(referenced_dataset_ids))
966
967     total = int(session.scalar(select(func.count()).select_from(stmt.subquery())) or 0)
968     task_count_subquery = (
969         select(AnalysisTask.dataset_id, func.count(AnalysisTask.id).label("task_count"))
970         .where(AnalysisTask.user_id == user.id)
971         .group_by(AnalysisTask.dataset_id)
972         .subquery()
973     )
974     sort_columns = {
975         "createdAt": Dataset.created_at,
976         "name": Dataset.name,
977         "sampleCount": Dataset.sample_count,
978         "baseCount": Dataset.base_cluster_count,
979         "taskCount": func.coalesce(task_count_subquery.c.task_count, 0),
980     }
981     sort_column = sort_columns.get(sort_by, Dataset.created_at)
982     ordering = asc(sort_column) if sort_order == "asc" else desc(sort_column)
983     page_stmt = (
984         stmt.outerjoin(task_count_subquery, task_count_subquery.c.dataset_id == Dataset.id)
985         .order_by(ordering, desc(Dataset.id))
986         .offset((page - 1) * page_size)
987         .limit(page_size)
988     )
989     datasets = session.scalars(page_stmt).all()
```

backend/app/datasets.py

```
990     qualities = {
991         quality.dataset_id: quality
992         for quality in session.scalars(
993             select(DatasetQuality).where(DatasetQuality.dataset_id.in_([dataset.id for dataset in datasets])),
994         ).all()
995     }
996     task_summaries = _task_summaries(session, user.id, [dataset.id for dataset in datasets])
997     total_pages = max(1, (total + page_size - 1) // page_size) if total else 0
998
999     return DatasetCatalogPageResponse(
1000         items=[
1001             _catalog_item(
1002                 session,
1003                 dataset,
1004                 quality=qualities.get(dataset.id),
1005                 task_summary=task_summaries.get(dataset.id),
1006             )
1007             for dataset in datasets
1008         ],
1009         total=total,
1010         page=page,
1011         pageSize=page_size,
1012         totalPages=total_pages,
1013     )
1014
1015
1016 @router.post("", response_model=DatasetCatalogItemResponse)
1017 async def upload_dataset(
1018     file: UploadFile = File(...),
1019     allow_duplicate: bool = Form(default=False, alias="allowDuplicate"),
1020     user: User = Depends(get_current_user),
1021     session: Session = Depends(get_session),
1022 ):
1023     """上传、解析、落盘和数据库写入构成一个业务事务，失败时同时清理两侧状态。"""
1024     staged: StagedUpload | None = None
1025     storage_path: Path | None = None
1026     try:
1027         staged = await _stage_upload(file)
1028         dataset_name = _dataset_name(staged.filename)
1029         if not allow_duplicate and _find_name_conflict(session, user, dataset_name):
1030             raise HTTPException(409, f"已存在同名数据集 "{dataset_name}"，请使用追加数据或确认保留独立副本")
1031
1032         _ensure_user_storage_quota(user.id, staged.size)
1033         parsed, _ = await asyncio.to_thread(_controlled_parse_mat_file, staged.path, staged.filename)
1034         file_hash, storage_path = await asyncio.to_thread(_save_staged_upload, user.id, staged)
1035
1036         dataset = Dataset(
1037             user_id=user.id,
1038             name=dataset_name,
1039             original_filename=staged.filename,
1040             storage_path=str(storage_path),
1041             file_hash=file_hash,
1042             sample_count=parsed["sampleCount"],
1043             base_cluster_count=parsed["baseCount"],
1044             has_ground_truth=parsed["hasLabels"],
1045             cluster_count=parsed["classCount"] if parsed["hasLabels"] else None,
1046             status="ready",
1047         )
1048         session.add(dataset)
1049         session.flush()
1050         quality = _upsert_quality(session, dataset, parsed)
1051         _record_revision(session, dataset, parsed, "uploaded", quality)
1052         session.flush()
1053         session.refresh(dataset)
1054         response = _catalog_item(session, dataset, parsed, quality)
1055         session.commit()
```

backend/app/datasets.py

```
1056     except Exception:
1057         session.rollback()
1058         if storage_path is not None:
1059             storage_path.unlink(missing_ok=True)
1060         raise
1061     finally:
1062         if staged is not None:
1063             staged.cleanup()
1064
1065     return response
1066
1067
1068 @router.put("/{dataset_id}", response_model=DatasetCatalogItemResponse)
1069 async def replace_dataset_file(
1070     dataset_id: int,
1071     file: UploadFile = File(...),
1072     allow_duplicate: bool = Form(default=False, alias="allowDuplicate"),
1073     user: User = Depends(get_current_user),
1074     session: Session = Depends(get_session),
1075 ):
1076     """保存旧版本快照后切换文件引用，提交失败时继续保留当前可用版本。"""
1077     dataset = _get_user_dataset(session, dataset_id, user)
1078     staged: StagedUpload | None = None
1079     storage_path: Path | None = None
1080     try:
1081         staged = await _stage_upload(file)
1082         next_name = _dataset_name(staged.filename)
1083         if not allow_duplicate and _find_name_conflict(session, user, next_name, exclude_dataset_id=dataset.id):
1084             raise HTTPException(409, f"已存在同名数据集“{next_name}”，请确认是否保留独立副本")
1085
1086         _ensure_user_storage_quota(user.id, staged.size)
1087         current_path = Path(dataset.storage_path)
1088         current_parsed, _ = await asyncio.to_thread(
1089             _controlled_parse_mat_file,
1090             current_path,
1091             dataset.original_filename,
1092         )
1093         current_quality = _quality_for_dataset(session, dataset, current_parsed)
1094         if _next_version(session, dataset.id) == 1:
1095             _record_revision(session, dataset, current_parsed, "uploaded", current_quality)
1096
1097         parsed, _ = await asyncio.to_thread(_controlled_parse_mat_file, staged.path, staged.filename)
1098         file_hash, storage_path = await asyncio.to_thread(_save_staged_upload, user.id, staged)
1099
1100         dataset.name = next_name
1101         dataset.original_filename = staged.filename
1102         dataset.storage_path = str(storage_path)
1103         dataset.file_hash = file_hash
1104         dataset.sample_count = parsed["sampleCount"]
1105         dataset.base_cluster_count = parsed["baseCount"]
1106         dataset.has_ground_truth = parsed["hasLabels"]
1107         dataset.cluster_count = parsed["classCount"] if parsed["hasLabels"] else None
1108         dataset.status = "ready"
1109         dataset.created_at = utc_now().replace(microsecond=0)
1110
1111         session.flush()
1112         quality = _upsert_quality(session, dataset, parsed)
1113         _record_revision(session, dataset, parsed, "replaced", quality)
1114         session.flush()
1115         session.refresh(dataset)
1116         response = _catalog_item(session, dataset, parsed, quality)
1117         session.commit()
1118     except Exception:
1119         session.rollback()
1120         if storage_path is not None:
1121             storage_path.unlink(missing_ok=True)
```

backend/app/datasets.py

```
1122         raise
1123     finally:
1124         if staged is not None:
1125             staged.cleanup()
1126
1127     return response
1128
1129
1130 @router.post("/{dataset_id}/append", response_model=DatasetCatalogItemResponse)
1131 async def append_dataset_file(
1132     dataset_id: int,
1133     file: UploadFile = File(...),
1134     user: User = Depends(get_current_user),
1135     session: Session = Depends(get_session),
1136 ):
1137     """校验新旧矩阵结构后生成完整新版本，不直接修改正在使用的原文件。"""
1138     dataset = _get_user_dataset(session, dataset_id, user)
1139     staged: StagedUpload | None = None
1140     output_staged: StagedUpload | None = None
1141     output_path: Path | None = None
1142     storage_path: Path | None = None
1143     try:
1144         staged = await _stage_upload(file)
1145         current_path = Path(dataset.storage_path)
1146         if not current_path.exists():
1147             raise HTTPException(409, "当前数据集文件不存在，无法追加")
1148
1149         current_parsed, _ = await asyncio.to_thread(
1150             _controlled_parse_mat_file,
1151             current_path,
1152             dataset.original_filename,
1153         )
1154         current_quality = _quality_for_dataset(session, dataset, current_parsed)
1155         if _next_version(session, dataset.id) == 1:
1156             _record_revision(session, dataset, current_parsed, "uploaded", current_quality)
1157
1158         output_file = tempfile.NamedTemporaryFile(prefix="soft-web-append-", suffix=".mat", delete=False)
1159         output_path = Path(output_file.name)
1160         output_file.close()
1161         parsed = await asyncio.to_thread(
1162             _build_appended_mat_file,
1163             current_path,
1164             dataset.original_filename,
1165             staged.path,
1166             staged.filename,
1167             output_path,
1168             _mat_limits(),
1169         )
1170         output_staged = StagedUpload(
1171             path=output_path,
1172             filename=dataset.original_filename,
1173             size=output_path.stat().st_size,
1174             file_hash=await asyncio.to_thread(_file_sha256, output_path),
1175         )
1176         _ensure_user_storage_quota(user.id, output_staged.size)
1177         file_hash, storage_path = await asyncio.to_thread(
1178             _save_staged_upload,
1179             user.id,
1180             output_staged,
1181             dataset.original_filename,
1182         )
1183
1184         dataset.storage_path = str(storage_path)
1185         dataset.file_hash = file_hash
1186         dataset.sample_count = parsed["sampleCount"]
1187         dataset.base_cluster_count = parsed["baseCount"]
```

backend/app/datasets.py

```

1188         dataset.has_ground_truth = parsed["hasLabels"]
1189         dataset.cluster_count = parsed["classCount"] if parsed["hasLabels"] else None
1190         dataset.status = "ready"
1191
1192         session.flush()
1193         quality = _upsert_quality(session, dataset, parsed)
1194         _record_revision(session, dataset, parsed, "appended", quality)
1195         session.flush()
1196         session.refresh(dataset)
1197         response = _catalog_item(session, dataset, parsed, quality)
1198         session.commit()
1199     except Exception:
1200         session.rollback()
1201         if storage_path is not None:
1202             storage_path.unlink(missing_ok=True)
1203         raise
1204     finally:
1205         if staged is not None:
1206             staged.cleanup()
1207         if output_staged is not None:
1208             output_staged.cleanup()
1209         elif output_path is not None:
1210             output_path.unlink(missing_ok=True)
1211
1212     return response
1213
1214
1215 @router.patch("/{dataset_id}", response_model=DatasetCatalogItemResponse)
1216 def rename_dataset(
1217     dataset_id: int,
1218     payload: DatasetRenameRequest,
1219     user: User = Depends(get_current_user),
1220     session: Session = Depends(get_session),
1221 ):
1222     dataset = _get_user_dataset(session, dataset_id, user)
1223     next_name = payload.name.strip()
1224     if not payload.allowDuplicate and _find_name_conflict(session, user, next_name, exclude_dataset_id=dataset.id):
1225         raise HTTPException(409, f"已存在同名数据集 "{next_name}"，请更换名称或确认保留重名副本")
1226
1227     dataset.name = next_name
1228     parsed = _parse_stored_dataset_file(dataset)
1229     quality = _quality_for_dataset(session, dataset, parsed)
1230     _record_revision(session, dataset, parsed, "renamed", quality)
1231     session.commit()
1232     session.refresh(dataset)
1233     return _catalog_item(session, dataset, parsed, quality)
1234
1235
1236 @router.post("/bulk-delete", response_model=MessageResponse)
1237 def bulk_delete_datasets(
1238     payload: DatasetBulkRequest,
1239     user: User = Depends(get_current_user),
1240     session: Session = Depends(get_session),
1241 ):
1242     datasets = session.scalars(
1243         select(Dataset).where(Dataset.user_id == user.id, Dataset.id.in_(payload.datasetIds)),
1244     ).all()
1245     if len(datasets) != len(set(payload.datasetIds)):
1246         raise HTTPException(404, "部分数据集不存在或无权访问")
1247
1248     cleanup_paths = _delete_datasets(session, datasets)
1249     process_pending_cleanup_jobs(limit=max(20, len(cleanup_paths)), bind=session.get_bind())
1250     return MessageResponse(message=f"已删除 {len(datasets)} 个数据集，相关文件已进入清理流程")
1251
1252
1253 @router.post("/export")

```

backend/app/datasets.py

```
1254 def export_datasets(  
1255     payload: DatasetBulkRequest,  
1256     user: User = Depends(get_current_user),  
1257     session: Session = Depends(get_session),  
1258 ):  
1259     datasets = session.scalars(  
1260         select(Dataset).where(Dataset.user_id == user.id, Dataset.id.in_(payload.datasetIds)),  
1261     ).all()  
1262     if len(datasets) != len(set(payload.datasetIds)):  
1263         raise HTTPException(404, "部分数据集不存在或无权访问")  
1264  
1265     archive = io.BytesIO()  
1266     with zipfile.ZipFile(archive, "w", zipfile.ZIP_DEFLATED) as zip_file:  
1267         for dataset in datasets:  
1268             storage_path = Path(dataset.storage_path)  
1269             if storage_path.exists():  
1270                 zip_file.write(storage_path, arcname=f"{dataset.name}_{dataset.id}{storage_path.suffix}")  
1271     archive.seek(0)  
1272     return StreamingResponse(  
1273         archive,  
1274         media_type="application/zip",  
1275         headers={"Content-Disposition": "attachment; filename=datasets-export.zip"},  
1276     )  
1277  
1278  
1279 def _revision_response(revision: DatasetRevision) -> DatasetRevisionResponse:  
1280     return DatasetRevisionResponse(  
1281         id=revision.id,  
1282         version=revision.version,  
1283         action=revision.action,  
1284         name=revision.name,  
1285         originalFilename=revision.original_filename,  
1286         createdAt=format_utc_iso(revision.created_at) or "",  
1287         sampleCount=revision.sample_count,  
1288         baseCount=revision.base_cluster_count,  
1289         classCount=revision.cluster_count or 0,  
1290         hasLabels=revision.has_ground_truth,  
1291         qualityStatus=revision.quality_status,  
1292         qualityIssues=_decode_issues(revision.quality_issues_json),  
1293     )  
1294  
1295  
1296 @router.get("/{dataset_id}/versions", response_model=list[DatasetRevisionResponse])  
1297 def list_dataset_versions(  
1298     dataset_id: int,  
1299     user: User = Depends(get_current_user),  
1300     session: Session = Depends(get_session),  
1301 ):  
1302     _get_user_dataset(session, dataset_id, user)  
1303     revisions = session.scalars(  
1304         select(DatasetRevision)  
1305         .where(DatasetRevision.dataset_id == dataset_id)  
1306         .order_by(DatasetRevision.version.desc()),  
1307     ).all()  
1308     return [_revision_response(revision) for revision in revisions]  
1309  
1310  
1311 @router.post("/{dataset_id}/quality", response_model=DatasetCatalogItemResponse)  
1312 def recheck_dataset_quality(  
1313     dataset_id: int,  
1314     user: User = Depends(get_current_user),  
1315     session: Session = Depends(get_session),  
1316 ):  
1317     dataset = _get_user_dataset(session, dataset_id, user)  
1318     parsed = _parse_stored_dataset_file(dataset)  
1319     quality = _upsert_quality(session, dataset, parsed)
```


backend/app/datasets.py

```
1320     session.commit()
1321     session.refresh(dataset)
1322     return _catalog_item(session, dataset, parsed, quality)
1323
1324
1325 @router.delete("/{dataset_id}", response_model=MessageResponse)
1326 def delete_dataset(
1327     dataset_id: int,
1328     user: User = Depends(get_current_user),
1329     session: Session = Depends(get_session),
1330 ):
1331     dataset = _get_user_dataset(session, dataset_id, user)
1332     cleanup_paths = _delete_datasets(session, [dataset])
1333     process_pending_cleanup_jobs(limit=max(20, len(cleanup_paths)), bind=session.get_bind())
1334     return MessageResponse(message="数据集已删除，相关文件已进入清理流程")
1335
1336
1337 @router.post("/parse", response_model=DatasetParseResponse)
1338 async def parse_dataset_file(
1339     file: UploadFile = File(...),
1340     user: User = Depends(get_current_user),
1341     session: Session = Depends(get_session),
1342 ):
1343     source_filename = file.filename or ""
1344     safe_filename = _safe_filename(source_filename) if source_filename else "unknown"
1345     staged: StagedUpload | None = None
1346
1347     try:
1348         staged = await _stage_upload(file)
1349         parsed, _ = await asyncio.to_thread(_controlled_parse_mat_file, staged.path, staged.filename)
1350     except HTTPException as exc:
1351         session.rollback()
1352         _persist_parse_log(
1353             session,
1354             user,
1355             filename=safe_filename,
1356             success=False,
1357             detail={"statusCode": exc.status_code, "errorType": type(exc).__name__},
1358         )
1359         logger.error("数据集文件解析失败: %s (状态码: %s)", safe_filename, exc.status_code)
1360         raise
1361     except Exception as exc:
1362         session.rollback()
1363         _persist_parse_log(
1364             session,
1365             user,
1366             filename=safe_filename,
1367             success=False,
1368             detail={"statusCode": 500, "errorType": type(exc).__name__},
1369         )
1370         logger.exception("数据集文件解析出现未预期异常: %s", safe_filename)
1371         raise HTTPException(500, "数据集解析失败") from exc
1372     else:
1373         _persist_parse_log(
1374             session,
1375             user,
1376             filename=safe_filename,
1377             success=True,
1378             detail={
1379                 "sampleCount": parsed.get("sampleCount", 0),
1380                 "baseCount": parsed.get("baseCount", 0),
1381                 "classCount": parsed.get("classCount", 0),
1382                 "hasLabels": bool(parsed.get("hasLabels")),
1383             },
1384         )
1385     return parsed
```

```
backend/app/datasets.py
```

```
1386     finally:
1387         if staged is not None:
1388             staged.cleanup()
1389
1390
1391 # ===== 文件: backend/app/models.py =====
1392 1 """业务数据模型及删除策略。
1393 2
1394 3 数据集被任务引用时禁止物理删除；任务删除后结果和导出记录由级联关系清理，磁盘文件另由清理队列处理。
1395 4 """
1396 5
1397 6 from datetime import datetime
1398 7
1399 8 from sqlalchemy import BigInteger, Boolean, Float, ForeignKey, Index, Integer, String, Text, UniqueConstraint, func
1400 9 from sqlalchemy.dialects import mysql
1401 10 from sqlalchemy.orm import Mapped, mapped_column, relationship
1402 11
1403 12 from .database import Base
1404 13 from .time_utils import UTCDateTime, utc_now
1405 14
1406 15
1407 16 # 既有 MySQL 库的用户和数据集主键为 BIGINT UNSIGNED；SQLite 测试保留 INTEGER 才能正常自增。
1408 17 IDENTIFIER_TYPE = (
1409 18     BigInteger()
1410 19     .with_variant(mysql.BIGINT(unsigned=True), "mysql")
1411 20     .with_variant(Integer(), "sqlite")
1412 21 )
1413 22
1414 23
1415 24 class User(Base):
1416 25     __tablename__ = "users"
1417 26
1418 27     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
1419 28     username: Mapped[str] = mapped_column(String(64), unique=True, index=True)
1420 29     email: Mapped[str | None] = mapped_column(String(255), unique=True, index=True, nullable=True)
1421 30     password_hash: Mapped[str] = mapped_column(String(255))
1422 31     role: Mapped[str] = mapped_column(String(32), default="user")
1423 32     status: Mapped[str] = mapped_column(String(32), default="active", index=True)
1424 33     last_login_at: Mapped[datetime | None] = mapped_column(UTCDateTime(), nullable=True)
1425 34     created_at: Mapped[datetime] = mapped_column(UTCDateTime(), default=utc_now, server_default=func.now())
1426 35     updated_at: Mapped[datetime] = mapped_column(
1427 36         UTCDateTime(),
1428 37         default=utc_now,
1429 38         server_default=func.now(),
1430 39         onupdate=utc_now,
1431 40     )
1432 41
1433 42     sessions: Mapped[list["UserSession"]] = relationship(back_populates="user")
1434 43     datasets: Mapped[list["Dataset"]] = relationship(back_populates="user")
1435 44
1436 45
1437 46 class EmailVerificationCode(Base):
1438 47     """邮箱验证码只保存哈希值，避免数据库泄漏时暴露仍在有效期内的明文验证码。"""
1439 48
1440 49     __tablename__ = "email_verification_codes"
1441 50
1442 51     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
1443 52     email: Mapped[str] = mapped_column(String(255), index=True)
1444 53     purpose: Mapped[str] = mapped_column(String(32), default="register", index=True)
1445 54     code_hash: Mapped[str] = mapped_column(String(255))
1446 55     client_ip: Mapped[str | None] = mapped_column(String(64), nullable=True)
1447 56     expires_at: Mapped[datetime] = mapped_column(UTCDateTime(), index=True)
1448 57     consumed_at: Mapped[datetime | None] = mapped_column(UTCDateTime(), nullable=True)
1449 58     created_at: Mapped[datetime] = mapped_column(
1450 59         UTCDateTime(), default=utc_now, server_default=func.now(), index=True,
1451 60     )
```

backend/app/models.py

```
61
62
63 class UserSession(Base):
64     __tablename__ = "user_sessions"
65
66     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
67     user_id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, ForeignKey("users.id", ondelete="CASCADE"), index=True)
68     refresh_token_hash: Mapped[str] = mapped_column(String(255), unique=True)
69     expires_at: Mapped[datetime] = mapped_column(UTCDateTime(), index=True)
70     revoked_at: Mapped[datetime | None] = mapped_column(UTCDateTime(), nullable=True)
71     created_at: Mapped[datetime] = mapped_column(UTCDateTime(), default=utc_now, server_default=func.now())
72
73     user: Mapped[User] = relationship(back_populates="sessions")
74
75
76 class Dataset(Base):
77     """用户上传数据集的当前版本, 历史文件由 DatasetRevision 单独保留。"""
78     __tablename__ = "datasets"
79     __table_args__ = (
80         Index("idx_datasets_user_created_at", "user_id", "created_at"),
81     )
82
83     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
84     user_id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, ForeignKey("users.id", ondelete="CASCADE"), index=True)
85     name: Mapped[str] = mapped_column(String(128))
86     original_filename: Mapped[str] = mapped_column(String(255))
87     storage_path: Mapped[str] = mapped_column(String(500))
88     file_hash: Mapped[str] = mapped_column(String(128))
89     sample_count: Mapped[int] = mapped_column(Integer)
90     base_cluster_count: Mapped[int] = mapped_column(Integer)
91     has_ground_truth: Mapped[bool] = mapped_column(Boolean, default=False)
92     cluster_count: Mapped[int | None] = mapped_column(Integer, nullable=True)
93     status: Mapped[str] = mapped_column(String(32), default="ready", index=True)
94     created_at: Mapped[datetime] = mapped_column(
95         UTCDateTime(), default=utc_now, server_default=func.now(), index=True,
96     )
97
98     user: Mapped[User] = relationship(back_populates="datasets")
99
100
101 class DatasetRevision(Base):
102     __tablename__ = "dataset_revisions"
103     __table_args__ = (
104         UniqueConstraint("dataset_id", "version", name="uk_dataset_revisions_dataset_version"),
105     )
106
107     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
108     dataset_id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, ForeignKey("datasets.id", ondelete="CASCADE"),
109         index=True)
110     version: Mapped[int] = mapped_column(Integer)
111     action: Mapped[str] = mapped_column(String(32))
112     name: Mapped[str] = mapped_column(String(128))
113     original_filename: Mapped[str] = mapped_column(String(255))
114     storage_path: Mapped[str] = mapped_column(String(500))
115     file_hash: Mapped[str] = mapped_column(String(128))
116     sample_count: Mapped[int] = mapped_column(Integer)
117     base_cluster_count: Mapped[int] = mapped_column(Integer)
118     has_ground_truth: Mapped[bool] = mapped_column(Boolean, default=False)
119     cluster_count: Mapped[int | None] = mapped_column(Integer, nullable=True)
120     quality_status: Mapped[str] = mapped_column(String(16), default="ready")
121     quality_issues_json: Mapped[str | None] = mapped_column(Text, nullable=True)
122     created_at: Mapped[datetime] = mapped_column(
123         UTCDateTime(), default=utc_now, server_default=func.now(), index=True,
124     )
125
126
```

backend/app/models.py

```
126 class DatasetQuality(Base):
127     __tablename__ = "dataset_qualities"
128
129     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
130     dataset_id: Mapped[int] = mapped_column(
131         IDENTIFIER_TYPE,
132         ForeignKey("datasets.id", ondelete="CASCADE"),
133         unique=True,
134         index=True,
135     )
136     status: Mapped[str] = mapped_column(String(16), default="ready", index=True)
137     issues_json: Mapped[str | None] = mapped_column(Text, nullable=True)
138     checked_at: Mapped[datetime] = mapped_column(
139         UTCDateTime(), default=utc_now, server_default=func.now(), onupdate=utc_now,
140     )
141
142
143 class AnalysisTask(Base):
144     """正式分析任务；承接 OMELET / OMELET-SV 的创建、调度、进度和结果关联。
145
146     worker_id 与 heartbeat_at 共同表示当前执行租约，避免多个调度实例重复领取同一任务。
147     """
148
149     __tablename__ = "analysis_tasks"
150     __table_args__ = (
151         Index("idx_analysis_tasks_user_dataset", "user_id", "dataset_id"),
152         Index("idx_analysis_tasks_dataset_status_finished", "dataset_id", "status", "finished_at"),
153         Index("idx_analysis_tasks_worker_id", "worker_id"),
154         Index("idx_analysis_tasks_heartbeat_at", "heartbeat_at"),
155         Index("idx_analysis_tasks_status_queued", "status", "queued_at", "id"),
156         Index("idx_analysis_tasks_status_heartbeat", "status", "heartbeat_at"),
157         Index("idx_analysis_tasks_user_status_queued", "user_id", "status", "queued_at"),
158     )
159
160     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
161     user_id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, ForeignKey("users.id", ondelete="CASCADE"), index=True)
162     dataset_id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, ForeignKey("datasets.id", ondelete="RESTRICT"),
163         index=True)
164     name: Mapped[str] = mapped_column(String(128), default="")
165     mode: Mapped[str] = mapped_column(String(32), default="OMELET-SV")
166     status: Mapped[str] = mapped_column(String(32), default="draft", index=True)
167     progress: Mapped[float] = mapped_column(Float, default=0.0)
168     current_run: Mapped[int] = mapped_column(Integer, default=0)
169     current_iter: Mapped[int] = mapped_column(Integer, default=0)
170     max_iter: Mapped[int] = mapped_column(Integer, default=20)
171     params_json: Mapped[str] = mapped_column(Text, default="{}")
172     error_message: Mapped[str | None] = mapped_column(Text, nullable=True)
173     failure_reason: Mapped[str | None] = mapped_column(String(64), nullable=True)
174     current_stage: Mapped[str | None] = mapped_column(String(64), nullable=True)
175     worker_id: Mapped[str | None] = mapped_column(String(128), nullable=True)
176     heartbeat_at: Mapped[datetime | None] = mapped_column(UTCDateTime(), nullable=True)
177     retry_count: Mapped[int] = mapped_column(Integer, default=0, server_default="0")
178     queued_at: Mapped[datetime | None] = mapped_column(UTCDateTime(), nullable=True, index=True)
179     started_at: Mapped[datetime | None] = mapped_column(UTCDateTime(), nullable=True)
180     finished_at: Mapped[datetime | None] = mapped_column(UTCDateTime(), nullable=True)
181     created_at: Mapped[datetime] = mapped_column(
182         UTCDateTime(), default=utc_now, server_default=func.now(), index=True,
183     )
184     updated_at: Mapped[datetime] = mapped_column(
185         UTCDateTime(),
186         default=utc_now,
187         server_default=func.now(),
188         onupdate=utc_now,
189     )
190
```

backend/app/models.py

```
191 class TaskResult(Base):
192     """任务唯一结果索引，矩阵和标签等大文件只在数据库中保存受控路径。"""
193     __tablename__ = "task_results"
194     __table_args__ = (
195         UniqueConstraint("task_id", name="uk_task_results_task_id"),
196     )
197
198     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
199     schema_version: Mapped[int | None] = mapped_column(Integer, nullable=True)
200     task_id: Mapped[int] = mapped_column(
201         IDENTIFIER_TYPE,
202         ForeignKey("analysis_tasks.id", ondelete="CASCADE"),
203         index=True,
204     )
205     metrics_json: Mapped[str | None] = mapped_column(Text, nullable=True)
206     kernel_weights_json: Mapped[str | None] = mapped_column(Text, nullable=True)
207     convergence_json: Mapped[str | None] = mapped_column(Text, nullable=True)
208     preview_json: Mapped[str | None] = mapped_column(Text, nullable=True)
209     labels_path: Mapped[str | None] = mapped_column(String(500), nullable=True)
210     ca_matrix_path: Mapped[str | None] = mapped_column(String(500), nullable=True)
211     s_matrix_path: Mapped[str | None] = mapped_column(String(500), nullable=True)
212     z_matrix_path: Mapped[str | None] = mapped_column(String(500), nullable=True)
213     runtime_seconds: Mapped[float | None] = mapped_column(Float, nullable=True)
214     created_at: Mapped[datetime] = mapped_column(UTCDateTime(), default=utc_now, server_default=func.now())
215
216
217 class TaskExport(Base):
218     """用户生成的交付档案记录，实际 ZIP 文件由存储清理队列维护生命周期。"""
219     __tablename__ = "task_exports"
220     __table_args__ = (
221         Index("idx_task_exports_task_id_id", "task_id", "id"),
222     )
223
224     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
225     task_id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, ForeignKey("analysis_tasks.id", ondelete="CASCADE"),
226     index=True)
227     export_type: Mapped[str] = mapped_column(String(32))
228     name: Mapped[str | None] = mapped_column(String(128), nullable=True)
229     items_json: Mapped[str | None] = mapped_column(Text, nullable=True)
230     status: Mapped[str] = mapped_column(String(32), default="ready")
231     filename: Mapped[str] = mapped_column(String(255))
232     storage_path: Mapped[str] = mapped_column(String(500))
233     file_size: Mapped[int | None] = mapped_column(Integer, nullable=True)
234     created_at: Mapped[datetime] = mapped_column(UTCDateTime(), default=utc_now, server_default=func.now())
235
236
237 class TaskTemplate(Base):
238     """用户保存的任务参数模板，便于重复创建同类分析。"""
239     __tablename__ = "task_templates"
240
241     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
242     user_id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, ForeignKey("users.id", ondelete="CASCADE"), index=True)
243     name: Mapped[str] = mapped_column(String(128))
244     mode: Mapped[str] = mapped_column(String(32), default="OMELET-SV")
245     params_json: Mapped[str] = mapped_column(Text, default="{}")
246     created_at: Mapped[datetime] = mapped_column(
247         UTCDateTime(), default=utc_now, server_default=func.now(), index=True,
248     )
249
250
251 class OperationLog(Base):
252     __tablename__ = "operation_logs"
253
254     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
255     user_id: Mapped[int | None] = mapped_column(IDENTIFIER_TYPE, ForeignKey("users.id", ondelete="SET NULL"),
```

backend/app/models.py

```
        nullable=True)
256     task_id: Mapped[int | None] = mapped_column(
257         IDENTIFIER_TYPE,
258         ForeignKey("analysis_tasks.id", ondelete="SET NULL"),
259         nullable=True,
260         index=True,
261     )
262     action: Mapped[str] = mapped_column(String(64), index=True)
263     level: Mapped[str] = mapped_column(String(16), default="info")
264     message: Mapped[str] = mapped_column(String(500))
265     detail_json: Mapped[str | None] = mapped_column(Text, nullable=True)
266     created_at: Mapped[datetime] = mapped_column(
267         UTCDateTime(), default=utc_now, server_default=func.now(), index=True,
268     )
269
270
271 class StorageCleanupJob(Base):
272     """数据库提交后异步删除磁盘文件的幂等队列任务。"""
273
274     __tablename__ = "storage_cleanup_jobs"
275     __table_args__ = (
276         Index("idx_storage_cleanup_jobs_status_retry", "status", "next_attempt_at", "id"),
277         Index("idx_storage_cleanup_jobs_created_at", "created_at"),
278         Index("idx_storage_cleanup_jobs_status", "status"),
279         Index("idx_storage_cleanup_jobs_next_attempt_at", "next_attempt_at"),
280         UniqueConstraint("storage_path", name="uk_storage_cleanup_jobs_storage_path"),
281     )
282
283     id: Mapped[int] = mapped_column(IDENTIFIER_TYPE, primary_key=True, autoincrement=True)
284     storage_path: Mapped[str] = mapped_column(String(500), nullable=False)
285     storage_kind: Mapped[str] = mapped_column(String(32), nullable=False)
286     status: Mapped[str] = mapped_column(String(16), default="pending")
287     attempts: Mapped[int] = mapped_column(Integer, default=0, server_default="0")
288     last_error: Mapped[str | None] = mapped_column(Text, nullable=True)
289     next_attempt_at: Mapped[datetime] = mapped_column(
290         UTCDateTime(), default=utc_now, server_default=func.now(),
291     )
292     created_at: Mapped[datetime] = mapped_column(
293         UTCDateTime(), default=utc_now, server_default=func.now(),
294     )
295     completed_at: Mapped[datetime | None] = mapped_column(UTCDateTime(), nullable=True)
296
```

```
front/src/appMeta.ts
```

```
// ===== 文件: front/src/appMeta.ts =====
1  /** 软件登记名称和版本的唯一来源，页面标题、页眉和截图材料统一引用。 */
2  export const SOFTWARE_NAME = '新材料数据分析---拓扑感知多核集成聚类分析系统';
3  export const SOFTWARE_SHORT_NAME = '新材料数据分析系统';
4  export const SOFTWARE_VERSION = 'V1.0';
5

// ===== 文件: front/src/App.tsx =====
1  import { useCallback, useEffect, useRef, useState } from 'react';
2  import {
3    ChevronLeft,
4    ChevronRight,
5    Logout,
6    Menu,
7    Play,
8    Upload,
9  } from 'lucide-react';
10 import { Navigate, NavLink, Route, Routes, useLocation, useNavigate } from 'react-router-dom';
11 import './landing.css';
12 import { SOFTWARE_SHORT_NAME } from './appMeta';
13 import sidebarBrandLogo from './images/Logo.svg';
14 import { LandingPage } from './LandingPage';
15 import { AuthFlipCard } from './AuthPages';
16 import { AppBackground } from './AppBackground';
17 import { DatasetManagementPage } from './DatasetManagementPage';
18 import { DataQualityPage } from './workbench/data-quality/DataQualityPage';
19 import { DatasetVersionsPage } from './workbench/dataset-versions/DatasetVersionsPage';
20 import { TaskCenterPage } from './workbench/tasks/TaskCenterPage';
21 import { TaskResultViews } from './workbench/results/TaskResultViews';
22 import { useTaskResult } from './workbench/results/useTaskResult';
23 import {
24   clearAuthSession,
25   logout as logoutFromApi,
26   restoreAuthSession,
27   subscribeAuthSession,
28   type AuthUser,
29 } from './api/auth';
30 import { publicFetch } from './api/client';
31 import {
32   dashboardGroups,
33   dashboardNavItems,
34   defaultWorkbenchPath,
35   getActiveWorkbenchSection,
36   legacyWorkbenchRedirects,
37 } from './dashboard/navigation';
38
39 const resultWorkbenchSections = new Set([
40   'analysis',
41   'ca-matrix',
42   'kernel-config',
43   'mkl',
44   'evaluation',
45   'visualization',
46   'results',
47   'export',
48   'reports',
49   'logs',
50 ]);
51
52 type BackendStatus = 'checking' | 'online' | 'offline';
53
```

front/src/App.tsx

```
54 function Sidebar({
55   activeSection,
56   collapsed,
57   open,
58   onClose,
59 }): {
60   activeSection: string;
61   collapsed: boolean;
62   open: boolean;
63   onClose: () => void;
64 }) {
65   const location = useLocation();
66   const taskId = new URLSearchParams(location.search).get('taskId');
67   const resultPaths = new Set([
68     '/workbench/analysis',
69     '/workbench/ca-matrix',
70     '/workbench/kernel-config',
71     '/workbench/mkl',
72     '/workbench/evaluation',
73     '/workbench/visualization',
74     '/workbench/results',
75     '/workbench/export',
76     '/workbench/reports',
77     '/workbench/logs',
78   ]);
79   return (
80     <>
81       <button
82         type="button"
83         className={`sidebar-backdrop ${open ? 'visible' : ''}`}
84         aria-label="关闭导航"
85         aria-hidden={!open}
86         tabIndex={open ? 0 : -1}
87         onClick={onClose}
88       />
89       <aside className={`sidebar ${open ? 'open' : ''} ${collapsed ? 'collapsed' : ''}`}>
90         <div className="sidebar-brand" title={SOFTWARE_SHORT_NAME}>
91           <span className="brand-logo" aria-hidden="true">
92             <img
93               className="sidebar-brand-logo-image"
94               src={sidebarBrandLogo}
95               width={30}
96               height={30}
97               alt=""
98               aria-hidden="true"
99             />
100           </span>
101           <span className="brand-copy">
102             <strong>{SOFTWARE_SHORT_NAME}</strong>
103             <small>新材料聚类分析</small>
104           </span>
105         </div>
106
107         <nav className="nav-list" aria-label="主导航">
108           {dashboardGroups.map((group) => (
109             <div className="nav-group" key={group.key}>
110               <span className="nav-group-label">{group.label}</span>
111               {dashboardNavItems
112                 .filter((item) => item.group === group.key)
113                 .map((item) => {
114                   const Icon = item.icon;
```


front/src/App.tsx

```
115         const isActive = activeSection === item.label;
116
117         return (
118             <NavLink
119                 key={item.label}
120                 to={resultPaths.has(item.path) && taskId ? ` ${item.path}?taskId=${encodeURIComponent(taskId)} ` :
121                 item.path}
122                 aria-current={isActive ? 'page' : undefined}
123                 className={({ isActive: routeIsActive }) =>
124                     `nav-item ${routeIsActive ? 'active' : ''}`
125                 }
126                 onClick={onClose}
127                 title={item.label}
128             >
129                 <span className="nav-icon" aria-hidden="true">
130                     <Icon size={20} />
131                 </span>
132                 <span className="nav-label">{item.label}</span>
133             </NavLink>
134         );
135     }}}
136 }
137 </nav>
138 </aside>
139 </>
140 );
141 }
142
143 function TopHeader({
144     activeSection,
145     currentUser,
146     onToggleMobileNav,
147     onLogout,
148 }): {
149     activeSection: string;
150     currentUser: AuthUser | null;
151     onToggleMobileNav: () => void;
152     onLogout: () => void;
153 }) {
154     const username = currentUser?.username || '研究用户';
155
156     return (
157         <header className="main-header">
158             <div className="navbar-left">
159                 <button
160                     type="button"
161                     className="header-icon-button mobile-nav-button"
162                     aria-label="打开导航"
163                     onClick={onToggleMobileNav}
164                 >
165                     <Menu size={18} />
166                 </button>
167                 <div className="header-context">
168                     <span>{SOFTWARE_SHORT_NAME} 工作台</span>
169                     <ChevronRight size={14} aria-hidden="true" />
170                     <div className="header-section-title" aria-label="当前模块">
171                         {activeSection}
172                     </div>
173                 </div>
174             </div>
```

front/src/App.tsx

```
175
176     <div className="navbar-right">
177       <div className="header-user" title={`当前用户: ${username}`} aria-label={`当前用户: ${username}`}>
178         <span className="header-user-avatar" aria-hidden="true">
179           {username.slice(0, 1).toUpperCase()}
180         </span>
181       </div>
182       <button
183         type="button"
184         className="header-icon-button"
185         aria-label="退出登录"
186         title="退出登录"
187         onClick={onLogout}>
188       >
189         <Logout size={16} aria-hidden="true" />
190     </button>
191   </div>
192 </header>
193 );
194 }
195
196 function StatusHeader({
197   status,
198   onImportData,
199   onCreateTask,
200 }): {
201   status: BackendStatus;
202   onImportData: () => void;
203   onCreateTask: () => void;
204 }) {
205   const statusText =
206     status === 'online' ? 'FastAPI 已连接' : status === 'offline' ? 'FastAPI 连接异常' : '正在检查 FastAPI...';
207   const statusClass = status === 'online' ? 'success' : status === 'offline' ? 'error' : 'warning';
208
209   return (
210     <section className="status-header" aria-label="系统连接状态">
211       <div className="status-left">
212         <span className={`status-dot ${statusClass}`} aria-hidden="true" />
213         <div>
214           <span className="status-label">Python 服务</span>
215           <strong>{statusText}</strong>
216         </div>
217       </div>
218       <div className="status-actions">
219         <button type="button" className="btn btn-secondary" onClick={onImportData}>
220           <Upload size={16} aria-hidden="true" />
221           导入数据
222         </button>
223         <button type="button" className="btn btn-primary" onClick={onCreateTask}>
224           <Play size={16} aria-hidden="true" />
225           创建任务
226         </button>
227       </div>
228     </section>
229   );
230 }
231
232 function WorkbenchPageContent({
233   activeSection,
234   sectionKey,
235   resultResource,
```

front/src/App.tsx

```
236   }: {
237     activeSection: string;
238     sectionKey: string;
239     resultResource: ReturnType<typeof useTaskResult>;
240   }) {
241     if (resultWorkbenchSections.has(sectionKey)) {
242       return <TaskResultViews section={sectionKey} resource={resultResource} />;
243     }
244
245     switch (activeSection) {
246       case '数据管理':
247         return <DatasetManagementPage />;
248       case '数据质量检查':
249         return <DataQualityPage />;
250       case '数据版本记录':
251         return <DatasetVersionsPage />;
252       case '任务中心':
253         return <TaskCenterPage />;
254       default:
255         return <Navigate to={defaultWorkbenchPath} replace />;
256     }
257   }
258
259   function App() {
260     const navigate = useNavigate();
261     const location = useLocation();
262     const workbenchSection = getActiveWorkbenchSection(location.pathname);
263     const workbenchKey = location.pathname.split('/')[2] || 'analysis';
264     const resultResource = useTaskResult(Boolean(workbenchSection) && resultWorkbenchSections.has(workbenchKey));
265     const activeSection = workbenchSection ?? '分析工作台';
266     const showStatusHeader = location.pathname === defaultWorkbenchPath;
267     const initialPathRef = useRef(location.pathname);
268     const sessionCheckedRef = useRef(false);
269     const [sidebarOpen, setSidebarOpen] = useState(false);
270     const [sidebarCollapsed, setSidebarCollapsed] = useState(false);
271     const [workbenchEntryPending, setWorkbenchEntryPending] = useState(false);
272     const [currentUser, setCurrentUser] = useState<AuthUser | null>(() => {
273       if (typeof localStorage === 'undefined') return null;
274       const storedUser = localStorage.getItem('soft_web_user');
275       if (!storedUser) return null;
276       try {
277         return JSON.parse(storedUser) as AuthUser;
278       } catch {
279         return null;
280       }
281     });
282     const [backendStatus, setBackendStatus] = useState<BackendStatus>('checking');
283
284     const openWorkbench = useCallback(() => {
285       navigate(defaultWorkbenchPath);
286       setSidebarOpen(false);
287     }, [navigate]);
288
289     useEffect(() => subscribeAuthSession((event) => {
290       setCurrentUser(null);
291       setSidebarOpen(false);
292
293       if (event.reason === 'logout') {
294         navigate('/', { replace: true });
295         return;
296       }
297     }

```

front/src/App.tsx

```
297
298   if (!['/login', '/register'].includes(location.pathname)) {
299     navigate('/login', { replace: true });
300   }
301 }, [location.pathname, navigate]);
302
303 const enterWorkbench = useCallback(async () => {
304   if (workbenchEntryPending) return;
305
306   setWorkbenchEntryPending(true);
307
308   try {
309     const user = await restoreAuthSession();
310     setCurrentUser(user);
311     openWorkbench();
312   } catch {
313     clearAuthSession();
314     setCurrentUser(null);
315     setSidebarOpen(false);
316     navigate('/login');
317   } finally {
318     setWorkbenchEntryPending(false);
319   }
320 }, [navigate, openWorkbench, workbenchEntryPending]);
321
322 const refreshBackendStatus = useCallback(async () => {
323   setBackendStatus('checking');
324   if (typeof fetch === 'undefined') {
325     setBackendStatus('offline');
326     return;
327   }
328   try {
329     const response = await publicFetch('/health');
330     if (!response.ok) {
331       throw new Error('health check failed');
332     }
333     setBackendStatus('online');
334   } catch {
335     setBackendStatus('offline');
336   }
337 }, []);
338
339 useEffect(() => {
340   if (sessionCheckedRef.current) {
341     return;
342   }
343
344   sessionCheckedRef.current = true;
345
346   const hasCachedUser = Boolean(localStorage.getItem('soft_web_user'));
347   const isWorkbenchPath = initialPathRef.current.startsWith('/workbench');
348   if (!hasCachedUser && !isWorkbenchPath) {
349     return;
350   }
351
352   let cancelled = false;
353
354   restoreAuthSession()
355     .then((user) => {
356       if (cancelled) return;
357       setCurrentUser(user);
```

front/src/App.tsx

```
358     if (['/', '/login', '/register'].includes(initialPathRef.current)) {
359       navigate(defaultWorkbenchPath, { replace: true });
360     }
361   })
362   .catch(() => {
363     if (cancelled) return;
364     clearAuthSession();
365     setCurrentUser(null);
366     if (isWorkbenchPath) {
367       navigate('/login', { replace: true });
368     }
369   });
370
371   return () => {
372     cancelled = true;
373   };
374 }, [navigate]);
375
376 useEffect(() => {
377   if (!workbenchSection) return;
378   void refreshBackendStatus();
379 }, [refreshBackendStatus, workbenchSection]);
380
381 const handleAuthSuccess = () => {
382   const storedUser = localStorage.getItem('soft_web_user');
383   if (storedUser) {
384     try {
385       setCurrentUser(JSON.parse(storedUser) as AuthUser);
386     } catch {
387       setCurrentUser(null);
388     }
389   }
390   openWorkbench();
391 };
392
393 const handleLogout = async () => {
394   await logoutFromApi();
395   setCurrentUser(null);
396   setSidebarOpen(false);
397   navigate('/', { replace: true });
398 };
399
400 const dashboardShell = workbenchSection ? (
401   <div className={`app-shell ${sidebarCollapsed ? 'sidebar-collapsed' : ''}`}>
402     <AppBackground />
403     <a className="skip-link" href="#main-content">
404       跳转到主要内容
405     </a>
406
407     <Sidebar
408       activeSection={activeSection}
409       collapsed={sidebarCollapsed}
410       open={sidebarOpen}
411       onClose={() => setSidebarOpen(false)}
412     />
413
414     <TopHeader
415       activeSection={activeSection}
416       currentUser={currentUser}
417       onToggleMobileNav={() => setSidebarOpen((value) => !value)}
418       onLogout={handleLogout}
```

front/src/App.tsx

```
419     />
420
421     <button
422       type="button"
423       className="sidebar-toggle-floating"
424       aria-label={sidebarCollapsed ? '展开侧栏' : '收起侧栏'}
425       title={sidebarCollapsed ? '展开侧栏' : '收起侧栏'}
426       onClick={() => setSidebarCollapsed((value) => !value)}
427     >
428       {sidebarCollapsed ? <ChevronRight size={15} /> : <ChevronLeft size={15} />}
429     </button>
430
431     <main className={`dashboard-content${workbenchKey === 'logs' ? ' is-log-layout' : ''}`} id="main-content">
432       {showStatusHeader ? (
433         <StatusHeader
434           status={backendStatus}
435           onImportData={() => navigate('/workbench/datasets')}
436           onCreateTask={() => navigate('/workbench/tasks')}
437         />
438       ) : null}
439       <WorkbenchPageContent
440         activeSection={activeSection}
441         sectionKey={workbenchKey}
442         resultResource={resultResource}
443       />
444     </main>
445   </div>
446 ) : (
447   <Navigate to={defaultWorkbenchPath} replace />
448 );
449
450 const isAuthRoute = location.pathname === '/login' || location.pathname === '/register';
451
452 return (
453   <>
454     <Routes>
455       <Route
456         path="/"
457         element={
458           <LandingPage
459             onLogin={() => navigate('/login')}
460             onRegister={() => navigate('/register')}
461             onEnterWorkbench={enterWorkbench}
462             isEnteringWorkbench={workbenchEntryPending}
463           />
464         }
465       />
466       <Route path="/login" element={null} />
467       <Route path="/register" element={null} />
468       <Route path="/workbench" element={
469         <Navigate to={defaultWorkbenchPath} replace />
470       } />
471       {Object.entries(legacyWorkbenchRedirects).map(([from, to]) => (
472         <Route key={from} path={from} element={
473           <Navigate to={to} replace />
474         } />
475       ))}
476       <Route path="/workbench/:section" element={dashboardShell} />
477       <Route path="*" element={
478         <Navigate to="/" replace />
479       } />
480     </Routes>
481     {isAuthRoute ? (
482       <AuthFlipCard
483         mode={location.pathname === '/register' ? 'register' : 'login'}
484         onSuccess={handleAuthSuccess}
485         onShowLogin={() => navigate('/login')}
486       />
487     ) : null}
488   </>
489 )
```

front/src/App.tsx

```
480         onShowRegister={() => navigate('/register')}
481         onBack={() => navigate('/')}}
482     />
483     ) : null}
484 </>
485 );
486 }
487
488 export default App;
489
// ===== 文件: front/src/api/auth-session.ts =====
1 /**
2  * 认证会话的前端生命周期管理。
3  * access token 只保存在内存中; refresh token 由后端通过 HttpOnly Cookie 管理, 页面不能读取或持久化它。
4  */
5 import { API_BASE_URL } from './config';
6 import {
7     ApiError,
8     createAuthenticationError,
9     createHttpError,
10    createNetworkError,
11    createTimeoutError,
12 } from './errors';
13
14 export const AUTH_STORAGE_KEYS = {
15     user: 'soft_web_user',
16 } as const;
17
18 const LEGACY_ACCESS_TOKEN_KEY = 'soft_web_access_token';
19 const LEGACY_REFRESH_TOKEN_KEY = 'soft_web_refresh_token';
20 const REFRESH_TIMEOUT_MS = 30_000;
21
22 export type AuthSessionEventReason = 'refresh_failed' | 'unauthorized' | 'logout';
23
24 export type AuthSessionEvent = {
25     reason: AuthSessionEventReason;
26     error?: ApiError;
27 };
28
29 type AuthSessionPayload = {
30     access_token: string;
31     user?: unknown;
32 };
33
34 type AuthSessionListener = (event: AuthSessionEvent) => void;
35
36 let accessToken: string | null = null;
37 let refreshPromise: Promise<string | null> | null = null;
38 let lastRefreshError: ApiError | null = null;
39 let logoutNotified = false;
40 const listeners = new Set<AuthSessionListener>();
41
42 function removeStoredAuth() {
43     if (typeof localStorage === 'undefined') return;
44     localStorage.removeItem(AUTH_STORAGE_KEYS.user);
45     localStorage.removeItem(LEGACY_ACCESS_TOKEN_KEY);
46     localStorage.removeItem(LEGACY_REFRESH_TOKEN_KEY);
47 }
48
49 function notifyLogout(reason: AuthSessionEventReason, error?: ApiError) {
```

front/src/api/auth-session.ts

```
50   if (logoutNotified) return;
51   logoutNotified = true;
52   const event = { reason, error } satisfies AuthSessionEvent;
53   listeners.forEach((listener) => listener(event));
54 }
55
56 export function subscribeAuthSession(listener: AuthSessionListener) {
57   listeners.add(listener);
58   return () => {
59     listeners.delete(listener);
60   };
61 }
62
63 export function getAccessToken() {
64   return accessToken;
65 }
66
67 export function setAccessToken(token: string | null) {
68   accessToken = token;
69   if (token) {
70     logoutNotified = false;
71     lastRefreshError = null;
72   }
73 }
74
75 export function saveAuthSession(auth: AuthSessionPayload) {
76   setAccessToken(auth.access_token);
77   if (typeof localStorage !== 'undefined' && auth.user !== undefined) {
78     localStorage.setItem(AUTH_STORAGE_KEYS.user, JSON.stringify(auth.user));
79   }
80   removeLegacyTokens();
81 }
82
83 function removeLegacyTokens() {
84   if (typeof localStorage === 'undefined') return;
85   localStorage.removeItem(LEGACY_ACCESS_TOKEN_KEY);
86   localStorage.removeItem(LEGACY_REFRESH_TOKEN_KEY);
87 }
88
89 export function getCachedAuthUser<T>() {
90   if (typeof localStorage === 'undefined') return null;
91   const storedUser = localStorage.getItem(AUTH_STORAGE_KEYS.user);
92   if (!storedUser) return null;
93   try {
94     return JSON.parse(storedUser) as T;
95   } catch {
96     return null;
97   }
98 }
99
100 export function cacheAuthUser(user: unknown) {
101   if (typeof localStorage === 'undefined') return;
102   localStorage.setItem(AUTH_STORAGE_KEYS.user, JSON.stringify(user));
103 }
104
105 export function clearAuthSession(options: {
106   notify?: boolean;
107   reason?: AuthSessionEventReason;
108   error?: ApiError;
109 } = {}) {
110   accessToken = null;
```



```
front/src/api/auth-session.ts
```

```
111   if (!options.error) {
112     lastRefreshError = null;
113   }
114   removeStoredAuth();
115   if (options.notify) {
116     notifyLogout(options.reason ?? 'logout', options.error);
117   }
118 }
119
120 function fetchRefreshToken() {
121   const controller = new AbortController();
122   let timedOut = false;
123   const timeoutId = setTimeout(() => {
124     timedOut = true;
125     controller.abort();
126   }, REFRESH_TIMEOUT_MS);
127
128   return (async () => {
129     try {
130       const response = await fetch(`${API_BASE_URL}/auth/refresh`, {
131         method: 'POST',
132         credentials: 'include',
133         signal: controller.signal,
134       });
135       const body = await response.json().catch(() => ({}));
136       if (!response.ok) {
137         throw createHttpError(response.status, body, '刷新登录状态失败');
138       }
139       if (!body || typeof body.access_token !== 'string' || !body.access_token) {
140         throw createAuthenticationError('刷新登录状态失败', body);
141       }
142       return body.access_token as string;
143     } catch (error: unknown) {
144       if (error instanceof ApiError) {
145         throw error;
146       }
147       if (timedOut) {
148         throw createTimeoutError(REFRESH_TIMEOUT_MS);
149       }
150       throw createNetworkError(error);
151     } finally {
152       clearTimeout(timeoutId);
153     }
154   })();
155 }
156
157 export async function refreshAccessToken(): Promise<string | null> {
158   // 刷新是跨请求的临界区，复用同一个 Promise 可避免多个 401 同时轮换 refresh token。
159   if (refreshPromise) {
160     return refreshPromise;
161   }
162
163   refreshPromise = (async () => {
164     try {
165       const token = await fetchRefreshToken();
166       setAccessToken(token);
167       return token;
168     } catch (error) {
169       lastRefreshError = error instanceof ApiError ? error : createNetworkError(error);
170       clearAuthSession({ notify: true, reason: 'refresh_failed', error: lastRefreshError });
171       return null;
172     }
173   })();
174 }
```

```
front/src/api/auth-session.ts
```

```
172     } finally {
173         refreshPromise = null;
174     }
175 })();
176
177 return refreshPromise;
178 }
179
180 export async function refreshAccessTokenOrThrow() {
181     const token = await refreshAccessToken();
182     if (token) return token;
183     throw lastRefreshError ?? createAuthenticationError();
184 }
185
186 export function notifyUnauthorized(error?: ApiError) {
187     clearAuthSession({ notify: true, reason: 'unauthorized', error });
188 }
189
```

```
// ===== 文件: front/src/api/client.ts =====
```

```
1 /**
2  * 前端统一请求边界：负责凭证注入、超时与取消、错误归类，以及 401 后的一次重试。
3  * 业务模块只描述接口路径和请求数据，不直接处理 Cookie、刷新并发或响应解析细节。
4  */
5 import { API_BASE_URL } from './config';
6 import {
7     ApiError,
8     createAbortedError,
9     createAuthenticationError,
10    createHttpError,
11    createNetworkError,
12    createTimeoutError,
13    toApiError,
14 } from './errors';
15 import {
16    getAccessToken,
17    notifyUnauthorized,
18    refreshAccessTokenOrThrow,
19 } from './auth-session';
20
21 export const DEFAULT_REQUEST_TIMEOUT_MS = 30_000;
22 export const DATASET_UPLOAD_TIMEOUT_MS = 5 * 60_000;
23 export const PROTECTED_DOWNLOAD_TIMEOUT_MS = 2 * 60_000;
24
25 export type ApiRequestOptions = RequestInit & {
26     timeoutMs?: number;
27     auth?: boolean;
28     retryOnUnauthorized?: boolean;
29     unauthenticatedMessage?: string;
30 };
31
32 function buildUrl(path: string) {
33     if (/^https?:\/\/\./i.test(path)) return path;
34     if (path === API_BASE_URL || path.startsWith(`${API_BASE_URL}/`)) return path;
35     return `${API_BASE_URL}${path.startsWith('/') ? path : `/${path}`}`;
36 }
37
38 function buildHeaders(options: RequestInit, token: string | null) {
39     const headers = new Headers(options.headers);
40     if (
41         options.body
```

```
front/src/api/client.ts
```

```
42     && typeof options.body === 'string'
43     && !headers.has('Content-Type')
44   ) {
45     headers.set('Content-Type', 'application/json');
46   }
47   if (token) {
48     headers.set('Authorization', `Bearer ${token}`);
49   }
50   return headers;
51 }
52
53 function createRequestSignal(timeoutMs: number, callerSignal?: AbortSignal) {
54   // 一个请求同时接受业务取消和超时取消，合并后的 signal 交给 fetch，避免悬挂请求占用页面资源。
55   const controller = new AbortController();
56   let timedOut = false;
57   const timeoutId = setTimeout(() => {
58     timedOut = true;
59     controller.abort();
60   }, timeoutMs);
61
62   const abortFromCaller = () => controller.abort();
63   if (callerSignal) {
64     if (callerSignal.aborted) {
65       controller.abort();
66     } else {
67       callerSignal.addEventListener('abort', abortFromCaller, { once: true });
68     }
69   }
70
71   return {
72     signal: controller.signal,
73     wasTimedOut: () => timedOut,
74     cleanup: () => {
75       clearTimeout(timeoutId);
76       callerSignal?.removeEventListener('abort', abortFromCaller);
77     },
78   };
79 }
80
81 async function fetchOnce(
82   path: string,
83   options: RequestInit,
84   token: string | null,
85   timeoutMs: number,
86 ) {
87   const requestSignal = createRequestSignal(timeoutMs, options.signal ?? undefined);
88   try {
89     return await fetch(buildUrl(path), {
90       ...options,
91       credentials: 'include',
92       headers: buildHeaders(options, token),
93       signal: requestSignal.signal,
94     });
95   } catch (error) {
96     if (requestSignal.wasTimedOut()) {
97       throw createTimeoutError(timeoutMs);
98     }
99     if (options.signal?.aborted) {
100       throw createAbortedError();
101     }
102     throw createNetworkError(error);
103   }
```

```
front/src/api/client.ts
```

```
103   } finally {
104     requestSignal.cleanup();
105   }
106 }
107
108 function isRefreshPath(path: string) {
109   return /\auth\/refresh(?:$|\?)/.test(path);
110 }
111
112 function throwIfAborted(signal?: AbortSignal) {
113   if (signal?.aborted) {
114     throw createAbortedError();
115   }
116 }
117
118 function withAuthenticationMessage(error: ApiError, message?: string) {
119   if (!message || error.category !== 'authentication') return error;
120   return new ApiError({
121     message,
122     status: error.status,
123     code: error.code,
124     category: error.category,
125     detail: error.detail,
126     isRetryable: error.isRetryable,
127   });
128 }
129
130 async function getTokenOrThrow(unauthenticatedMessage?: string) {
131   const existingToken = getAccessToken();
132   if (existingToken) return existingToken;
133
134   try {
135     return await refreshAccessTokenOrThrow();
136   } catch (error) {
137     throw withAuthenticationMessage(toApiError(error), unauthenticatedMessage);
138   }
139 }
140
141 export async function apiFetch(path: string, options: ApiRequestOptions = {}) {
142   const {
143     timeoutMs = DEFAULT_REQUEST_TIMEOUT_MS,
144     auth = true,
145     retryOnUnauthorized = true,
146     unauthenticatedMessage,
147     ...requestInit
148   } = options;
149
150   throwIfAborted(requestInit.signal ?? undefined);
151
152   // refresh 端点永远走无认证分支，避免没有 access token 时再次递归 refresh。
153   if (!auth || isRefreshPath(path)) {
154     return fetchOnce(path, requestInit, null, timeoutMs);
155   }
156
157   const token = await getTokenOrThrow(unauthenticatedMessage);
158   const response = await fetchOnce(path, requestInit, token, timeoutMs);
159   if (
160     response.status !== 401
161     || !retryOnUnauthorized
162     || isRefreshPath(path)
163   ) {
```

```
front/src/api/client.ts
```

```
164     return response;
165   }
166
167   throwIfAborted(requestInit.signal ?? undefined);
168   // 所有并发请求共享 auth-session 中的刷新 Promise; 当前请求只在刷新成功后重试一次。
169   let refreshedToken: string;
170   try {
171     refreshedToken = await refreshAccessTokenOrThrow();
172   } catch (error) {
173     const apiError = toApiError(error);
174     if (apiError.category === 'authentication') {
175       throw new ApiError({
176         message: '登录已过期, 请重新登录',
177         status: 401,
178         code: 'AUTHENTICATION_EXPIRED',
179         category: 'authentication',
180         detail: apiError.detail,
181       });
182     }
183     throw apiError;
184   }
185
186   throwIfAborted(requestInit.signal ?? undefined);
187   const retriedResponse = await fetchOnce(path, requestInit, refreshedToken, timeoutMs);
188   if (retriedResponse.status === 401) {
189     const error = createAuthenticationError('登录已过期, 请重新登录');
190     notifyUnauthorized(error);
191   }
192   return retriedResponse;
193 }
194
195 async function readResponseBody(response: Response): Promise<unknown> {
196   if (response.status === 204) return undefined;
197
198   if (typeof response.json === 'function') {
199     try {
200       return await response.json();
201     } catch {
202       // 某些下载或空响应没有 JSON 正文, 继续尝试文本读取。
203     }
204   }
205
206   if (typeof response.text === 'function') {
207     let text: string;
208     try {
209       text = await response.text();
210     } catch {
211       return undefined;
212     }
213     if (!text) return undefined;
214     try {
215       return JSON.parse(text) as unknown;
216     } catch {
217       return text;
218     }
219   }
220
221   return undefined;
222 }
223
224 export async function apiJson<T>(path: string, options: ApiRequestOptions = {}) {
```

```
front/src/api/client.ts
```

```
225   const response = await apiFetch(path, options);
226   const body = await readResponseBody(response);
227   if (!response.ok) {
228     throw createHttpError(response.status, body);
229   }
230   return body as T;
231 }
232
233 export async function ensureApiResponse(response: Response, fallback = '请求失败') {
234   if (response.ok) return response;
235   const body = await readResponseBody(response);
236   throw createHttpError(response.status, body, fallback);
237 }
238
239 export async function apiBlob(path: string, options: ApiRequestOptions = {}) {
240   const response = await apiFetch(path, options);
241   if (!response.ok) {
242     const body = await readResponseBody(response);
243     throw createHttpError(response.status, body);
244   }
245   return response.blob();
246 }
247
248 export function publicFetch(path: string, options: Omit<ApiRequestOptions, 'auth'> = {}) {
249   return apiFetch(path, { ...options, auth: false });
250 }
251
252 export function publicJson<T>(path: string, options: Omit<ApiRequestOptions, 'auth'> = {}) {
253   return apiJson<T>(path, { ...options, auth: false });
254 }
255
```

```
// ===== 文件: front/src/workbench/tasks/components/TaskCreateDrawer.tsx =====
```

```
1 import { useEffect, useMemo, useState } from 'react';
2 import { createPortal } from 'react-dom';
3 import { CheckCircle2, ChevronLeft, ChevronRight, Database, FileStack, Settings2, Trash2 } from 'lucide-react';
4 import { SelectField } from '../../../components/SelectField';
5 import type { CreateTaskPayload, DatasetOption, TaskMode, TaskParams, TaskTemplate } from '../types';
6
7 type CreateStep = 1 | 2 | 3;
8
9 type Props = {
10   open: boolean;
11   datasets: DatasetOption[];
12   templates: TaskTemplate[];
13   initialDatasetId?: number | null;
14   submitting?: boolean;
15   onClose: () => void;
16   onSubmit: (payload: CreateTaskPayload, saveAsTemplateName?: string) => Promise<void> | void;
17   onDeleteTemplate: (template: TaskTemplate) => void;
18 };
19
20 const DEFAULT_PARAMS: TaskParams = {
21   nBase: 20,
22   sigma: 1,
23   lambda: 5,
24   gamma: 5,
25   anchor: 10,
26   runs: 10,
27   maxIter: 10,
28   randomSeed: 1,
```

```
front/src/workbench/tasks/components/TaskCreateDrawer.tsx
```

```
29 };
30
31 const STEPS: Array<{ key: CreateStep; label: string; icon: typeof Database }> = [
32   { key: 1, label: '选择数据', icon: Database },
33   { key: 2, label: '配置算法', icon: Settings2 },
34   { key: 3, label: '确认执行', icon: CheckCircle2 },
35 ];
36
37 export function TaskCreateDrawer({
38   open,
39   datasets,
40   templates,
41   initialDatasetId,
42   submitting,
43   onClose,
44   onSubmit,
45   onDeleteTemplate,
46 }: Props) {
47   const [step, setStep] = useState<CreateStep>(1);
48   const [name, setName] = useState('');
49   const [mode, setMode] = useState<TaskMode>('OMELET-SV');
50   const [datasetId, setDatasetId] = useState('');
51   const [params, setParams] = useState<TaskParams>(DEFAULT_PARAMS);
52   const [startImmediately, setStartImmediately] = useState(true);
53   const [templateId, setTemplateId] = useState('');
54   const [templateName, setTemplateName] = useState('');
55   const [error, setError] = useState('');
56
57   const selectedDataset = useMemo(
58     () => datasets.find((item) => String(item.id) === datasetId) || null,
59     [datasets, datasetId],
60   );
61
62   useEffect(() => {
63     if (!open) return;
64     setStep(1);
65     setError('');
66     setTemplateName('');
67     if (initialDatasetId) {
68       setDatasetId(String(initialDatasetId));
69     } else if (datasets[0]) {
70       setDatasetId(String(datasets[0].id));
71     }
72   }, [open, initialDatasetId, datasets]);
73
74   useEffect(() => {
75     if (!selectedDataset) return;
76     setParams((prev) => ({ ...prev, nBase: Math.min(prev.nBase || 20, selectedDataset.baseCount || 20) }));
77     if (!name.trim()) setName(`${selectedDataset.name} ${mode} 任务`);
78   }, [selectedDataset?.id, mode]);
79
80   function applyTemplate(id: string) {
81     setTemplateId(id);
82     const template = templates.find((item) => String(item.id) === id);
83     if (!template) return;
84     setMode((template.mode as TaskMode) || 'OMELET-SV');
85     setParams({
86       ...DEFAULT_PARAMS,
87       ...template.params,
88       lambda: Number(template.params.lambda ?? DEFAULT_PARAMS.lambda),
89       nBase: Number(template.params.nBase ?? DEFAULT_PARAMS.nBase),
```

```
front/src/workbench/tasks/components/TaskCreateDrawer.tsx
```

```
90     sigma: Number(template.params.sigma ?? DEFAULT_PARAMS.sigma),
91     gamma: Number(template.params.gamma ?? DEFAULT_PARAMS.gamma),
92     anchor: Number(template.params.anchor ?? DEFAULT_PARAMS.anchor),
93     runs: Number(template.params.runs ?? DEFAULT_PARAMS.runs),
94     maxIter: Number(template.params.maxIter ?? DEFAULT_PARAMS.maxIter),
95     randomSeed: Number(template.params.randomSeed ?? DEFAULT_PARAMS.randomSeed),
96   });
97 }
98
99 function validateData() {
100   if (!datasetId) {
101     setError('请选择数据集');
102     return false;
103   }
104   if (selectedDataset?.qualityStatus === 'error') {
105     setError('该数据集质量检查未通过，无法创建任务');
106     return false;
107   }
108   if (selectedDataset?.hasLabels === false || (selectedDataset?.classCount ?? 0) < 2) {
109     setError('OMELET 分析需要包含至少两个类别的 y 标签');
110     return false;
111   }
112   return true;
113 }
114
115 function validateParams() {
116   if (!selectedDataset) return false;
117   if (!Number.isFinite(params.nBase) || params.nBase < 1 || params.nBase > selectedDataset.baseCount) {
118     setError('n_base 应为 1 到 ${selectedDataset.baseCount} 之间的整数');
119     return false;
120   }
121   if ([params.sigma, params.lambda, params.gamma, params.runs, params.maxIter].some((value) =>
122     !Number.isFinite(value) || value <= 0)) {
123     setError('sigma、lambda、gamma、runs 与 max_iter 必须大于 0');
124     return false;
125   }
126   if (!Number.isInteger(params.randomSeed) || params.randomSeed < 0) {
127     setError('random_seed 必须是大于或等于 0 的整数');
128     return false;
129   }
130   if (mode === 'OMELET-SV' && (!Number.isFinite(params.anchor) || params.anchor < 1)) {
131     setError('OMELET-SV 的 anchor 必须大于 0');
132     return false;
133   }
134   return true;
135 }
136
137 function nextStep() {
138   setError('');
139   if (step === 1 && !validateData()) return;
140   if (step === 2 && !validateParams()) return;
141   setStep((current) => Math.min(3, current + 1) as CreateStep);
142 }
143
144 async function handleSubmit() {
145   setError('');
146   if (!validateData() || !validateParams()) return;
147
148   const payload: CreateTaskPayload = {
149     datasetId: Number(datasetId),
150     name: name.trim() || undefined,
```



```
front/src/workbench/tasks/components/TaskCreateDrawer.tsx
```

```
150     mode,
151     startImmediately,
152     templateId: templateId ? Number(templateId) : undefined,
153     params: {
154       nBase: Number(params.nBase),
155       sigma: Number(params.sigma),
156       lambda: Number(params.lambda),
157       gamma: Number(params.gamma),
158       anchor: mode === 'OMELET-SV' ? Number(params.anchor) : 0,
159       runs: Number(params.runs),
160       maxIter: Number(params.maxIter),
161       randomSeed: Number(params.randomSeed),
162     },
163   };
164
165   try {
166     await onSubmit(payload, templateName.trim() || undefined);
167   } catch (err) {
168     setError(err instanceof Error ? err.message : '创建失败');
169   }
170 }
171
172 if (!open) return null;
173
174 return createPortal(
175   <div className="task-drawer-root" role="dialog" aria-modal="true" aria-label="新建任务">
176     <button type="button" className="task-drawer-mask" aria-label="关闭新建任务" onClick={onClose} />
177     <aside className="task-drawer-panel create">
178       <header className="task-drawer-header">
179         <div>
180           <h2>新建分析任务</h2>
181           <span>将数据集、算法参数和执行计划整理为可追踪的实验任务。</span>
182         </div>
183         <button type="button" className="btn btn-secondary" onClick={onClose}>关闭</button>
184       </header>
185
186       <div className="task-create-stepper" aria-label="创建任务步骤">
187         {STEPS.map((item) => {
188           const Icon = item.icon;
189           return (
190             <div key={item.key} className={step === item.key ? 'active' : step > item.key ? 'done' : ''}>
191               <span><Icon size={15} aria-hidden="true" /></span>
192               <strong>{item.key} {item.label}</strong>
193             </div>
194           );
195         })}
196       </div>
197
198       <div className="task-drawer-body">
199         {step === 1 ? (
200           <div>
201             <label className="task-form-field">
202               <span>任务名称</span>
203               <input value={name} onChange={(event) => setName(event.target.value)} placeholder="请输入便于识别的任务名称" />
204             </label>
205
206             <SelectField
207               className="task-form-field"
208               label="选择数据集"
209               value={datasetId}
210               options=[
```

```
front/src/workbench/tasks/components/TaskCreateDrawer.tsx
```

```
211         { value: '', label: '请选择数据集' },
212         ...datasets.map((item) => ({
213             value: String(item.id),
214             label: `${item.name} (m=${item.baseCount}, n=${item.sampleCount})${item.hasLabels ?
`, ${item.classCount} 类标签` : ', 无标签'})${item.qualityStatus === 'error' ? ', 质量异常' : ''})`),
215         )),
216     ]}
217     onChange={setDatasetId}
218 />
219
220     {selectedDataset ? (
221         <section className={`task-dataset-snapshot quality-${selectedDataset.qualityStatus}`}>
222             <header><Database size={16} aria-hidden="true" /><strong>{selectedDataset.name}</strong></header>
223             <dl>
224                 <div><dt>样本数量</dt><dd>{selectedDataset.sampleCount}</dd></div>
225                 <div><dt>基础聚类</dt><dd>{selectedDataset.baseCount}</dd></div>
226                 <div><dt>质量状态</dt><dd>{selectedDataset.qualityStatus === 'error' ? '需处理' :
selectedDataset.qualityStatus === 'warning' ? '待确认' : '检查通过'}</dd></div>
227             </dl>
228             {selectedDataset.qualityIssues?.length ? <p>{selectedDataset.qualityIssues.join('; ')}</p> : null}
229         </section>
230     ) : null}
231
232     <section className="task-template-picker">
233         <div className="task-template-picker-heading"><FileStack size={16} aria-hidden="true"
/><strong>参数模板</strong></div>
234         <SelectField
235             className="task-form-field"
236             label="从模板填充"
237             value={templateId}
238             options=[
239                 { value: '', label: '不使用模板' },
240                 ...templates.map((item) => ({
241                     value: String(item.id),
242                     label: `${item.name} (${item.mode})`,
243                 })),
244             ]}
245             onChange={applyTemplate}
246         />
247         {templates.length > 0 ? (
248             <div className="task-template-list">
249                 {templates.map((item) => (
250                     <div key={item.id}>
251                         <span>{item.name} • {item.mode}</span>
252                         <button type="button" title="删除模板" aria-label={`删除模板 ${item.name}`} onClick={() =>
onDeleteTemplate(item)}>
253                             <Trash2 size={14} aria-hidden="true" />
254                         </button>
255                     </div>
256                 ))}
257             </div>
258         ) : <small>尚无已保存模板，可在最后一步保存当前参数。</small>)}
259     </section>
260 </>
261 ) : null}
262
263 {step === 2 ? (
264     <>
265         <section className="task-mode-control" aria-label="算法模式">
266             {[ 'OMELET', 'OMELET-SV' ] as TaskMode[]}.map((item) => (
267                 <button key={item} type="button" className={mode === item ? 'active' : ''} onClick={() =>
```

```
front/src/workbench/tasks/components/TaskCreateDrawer.tsx
```

```
        setMode(item))>{item}</button>
268      )))
269    </section>
270    <p className="task-parameter-note">OMELET-SV 适用于大规模数据并启用 anchor；OMELET 使用完整相似性计算。</p>
271    <div className="task-create-grid">
272      {(
273        [
274          ['nBase', 'n_base', 1, 1],
275          ['sigma', 'sigma', 0.1, 0.1],
276          ['lambda', 'lambda', 0.1, 0.1],
277          ['gamma', 'gamma', 0.1, 0.1],
278          ['runs', 'runs', 1, 1],
279          ['maxIter', 'max_iter', 1, 1],
280          ['randomSeed', 'random_seed', 0, 1],
281        ] as const
282      ).map(([key, label, min, stepValue]) => (
283        <label key={key} className="task-form-field">
284          <span>{label}</span>
285          <input type="number" min={min} step={stepValue} value={params[key]} onChange={(event) =>
setParams((prev) => ({ ...prev, [key]: Number(event.target.value) }))) />
286        </label>
287      )))
288    {mode === 'OMELET-SV' ? (
289      <label className="task-form-field">
290        <span>anchor</span>
291        <input type="number" min={1} step={1} value={params.anchor} onChange={(event) => setParams((prev)
=> ({ ...prev, anchor: Number(event.target.value) }))) />
292      </label>
293    ) : null}
294    </div>
295    </>
296    ) : null}
297
298    {step === 3 ? (
299      <>
300        <section className="task-create-summary">
301          <header><CheckCircle2 size={17} aria-hidden="true" /><strong>执行摘要</strong></header>
302          <dl>
303            <div><dt>任务</dt><dd>{name || '未命名任务'}</dd></div>
304            <div><dt>数据集</dt><dd>{selectedDataset?.name || '-'}</dd></div>
305            <div><dt>算法</dt><dd>{mode}</dd></div>
306            <div><dt>运行次数</dt><dd>{params.runs}</dd></div>
307            <div><dt>迭代上限</dt><dd>{params.maxIter}</dd></div>
308            <div><dt>基础聚类</dt><dd>{params.nBase}</dd></div>
309          </dl>
310        </section>
311        <label className="task-check-field">
312          <input type="checkbox" checked={startImmediately} onChange={(event) =>
setStartImmediately(event.target.checked)} />
313          <span>创建后立即启动</span>
314        </label>
315        <label className="task-form-field">
316          <span>同时保存为模板（可选）</span>
317          <input value={templateName} onChange={(event) => setTemplateName(event.target.value)} placeholder="例如
默认 OMELET-SV 参数" />
318        </label>
319      </>
320    ) : null}
321
322    {error ? <div className="task-error-box">{error}</div> : null}
323  </div>
```

```
front/src/workbench/tasks/components/TaskCreateDrawer.tsx
```

```
324
325     <footer className="task-drawer-footer">
326       {step > 1 ? (
327         <button type="button" className="btn btn-secondary" onClick={() => setStep((current) => Math.max(1,
328           current - 1) as CreateStep)}>
329         <ChevronLeft size={16} aria-hidden="true" /> 上一步
330       ) : null}
331     <button type="button" className="btn btn-secondary" onClick={onClose}>取消</button>
332     {step < 3 ? (
333       <button type="button" className="btn btn-primary" onClick={nextStep}>
334       下一步 <ChevronRight size={16} aria-hidden="true" />
335     </button>
336   ) : (
337     <button type="button" className="btn btn-primary" disabled={!submitting} onClick={handleSubmit}>
338     {submitting ? '提交中...' : startImmediately ? '创建并启动任务' : '创建任务草稿'}
339   </button>
340   )}
341 </footer>
342 </aside>
343 </div>,
344 document.body,
345 );
346 }
347
```

```
// ===== 文件: front/src/workbench/results/TaskResultViews.tsx =====
```

```
1 /**
2  * 结果页面只消费任务结果接口返回的 envelope，不在前端生成演示指标、矩阵或运行日志。
3  * 状态页处理未完成任务，完成后的 section 再交给对应的真实结果视图。
4  */
5 import { useMemo, useState } from 'react';
6 import { useNavigate } from 'react-router-dom';
7 import {
8   Activity,
9   Award,
10   BarChart3,
11   CheckCircle2,
12   ClipboardCheck,
13   Clock3,
14   Download,
15   Gauge,
16   Info,
17   RefreshCw,
18   Target,
19   XCircle,
20 } from 'lucide-react';
21 import { retryTask } from '../../api/tasks';
22 import { SelectField } from '../../components/SelectField';
23 import {
24   WorkbenchNotice,
25   WorkbenchPageHeader,
26   WorkbenchSectionHeader,
27   WorkbenchStatus,
28 } from '../../WorkbenchUi';
29 import { getTaskRunProgress } from '../../tasks/taskStatus';
30 import { AnalysisOverviewPage } from './AnalysisOverviewPage';
31 import {
32   CaResultPage,
33   ExportResultPage,
34   KernelResultPage,
```

front/src/workbench/results/TaskResultViews.tsx

```
35   MklResultPage,
36   ReportResultPage,
37   ResultsAnalysisPage,
38   VisualizationResultPage,
39 } from './ResultDetailPages';
40 import {
41   buildEvaluationTrendOption,
42   type EvaluationMetricKey,
43   type EvaluationTrendMode,
44   useResultChart,
45 } from './ResultChartPanels';
46 import { MetricStrip, ReadyHeader, ResultBackButton, useShouldShowResultBack } from './ResultPageShared';
47 import {
48   buildEvaluationCsv,
49   deriveEvaluationSummary,
50   evaluationMetricKeys,
51 } from './evaluationPresentation';
52 import {
53   downloadTextFile,
54   formatNumber,
55   formatPercent,
56   metricLabels,
57 } from './resultPresentation';
58 import { TaskLogsPage } from './TaskLogsPage';
59 import type { TaskResultResource } from './types';
60
61 type Props = {
62   section: string;
63   resource: TaskResultResource;
64 };
65
66 function ResultState({ resource }: { resource: TaskResultResource }) {
67   const navigate = useNavigate();
68   const showBackButton = useShouldShowResultBack();
69   const envelope = resource.envelope;
70   const task = envelope?.task;
71   const state = envelope?.state;
72
73   const retry = async () => {
74     if (!task) return;
75     await retryTask(task.id);
76     resource.refresh();
77   };
78
79   let title = '正在加载分析结果';
80   let detail = '正在读取任务、结果和可视化预览。';
81   let tone: 'info' | 'warning' | 'error' | 'success' = 'info';
82
83   if (resource.error) {
84     title = '结果加载失败';
85     detail = resource.error;
86     tone = 'error';
87   }
88   if (state === 'empty') {
89     title = '还没有可用的分析结果';
90     detail = '请先创建并完成一个带 y 标签的数据集分析任务。';
91   }
92   if (state === 'draft') {
93     title = '任务尚未启动';
94     detail = '启动任务后，这里会显示计算过程和结果。';
95   }
```

```
front/src/workbench/results/TaskResultViews.tsx
```

```
96   if (state === 'queued') {
97     title = '任务正在排队';
98     detail = `队列位置 ${task?.queuePosition ?? '—'}, 轮到后会自动开始。`;
99     tone = 'warning';
100  }
101  if (state === 'running' && task) {
102    const run = getTaskRunProgress(task);
103    title = 'OMELET 正在运行';
104    detail = `第 ${run.current}/${run.total} 轮, 迭代 ${task.currentIter ?? 0}/${task.maxIter ?? 0}, ${Math.round(task.progress ?? 0)}%。`;
105    tone = 'warning';
106  }
107  if (state === 'failed') {
108    title = '任务执行失败';
109    detail = task?.errorMessage || '算法子进程未能完成, 请查看日志后重试。';
110    tone = 'error';
111  }
112  if (state === 'cancelled') {
113    title = '任务已取消';
114    detail = '该任务没有完整结果, 可以调整参数后重新执行。';
115    tone = 'warning';
116  }
117  if (state === 'legacy') {
118    title = '结果结构需要更新';
119    detail = '当前任务结果缺少完整算法产物, 重新执行任务后可生成新版分析结果。';
120    tone = 'warning';
121  }
122
123  return (
124    <section className="soft-page result-state-page" aria-live="polite">
125      <WorkbenchPageHeader
126        icon={Activity}
127        title="分析结果"
128        context={task ? `任务 #${task.id} • ${task.datasetName} • ${task.mode}` : '结果上下文'}
129        status={<WorkbenchStatus tone={tone} pulse={state === 'running'}>{title}</WorkbenchStatus>}
130        backAction={showBackButton ? <ResultBackButton taskId={task?.id} /> : undefined}
131      />
132      <section className="panel result-state-panel">
133        <WorkbenchNotice tone={tone} icon={tone === 'error' ? XCircle : Activity} title={title} detail={detail} />
134        <div className="result-state-actions">
135          {(state === 'failed' || state === 'cancelled' || state === 'legacy') && task ? (
136            <button type="button" className="btn btn-primary" onClick={() => void retry()}>重新执行</button>
137          ) : null}
138          <button type="button" className="btn btn-secondary" onClick={() => navigate('/workbench/tasks')}>
139            {state === 'empty' ? '创建任务' : '打开任务中心'}
140          </button>
141          {resource.error ? (
142            <button type="button" className="btn btn-secondary" onClick={resource.refresh}>
143              <RefreshCw size={15} />重试加载
144            </button>
145          ) : null}
146        </div>
147      </section>
148    </section>
149  );
150 }
151
152 function EvaluationPage({ resource }: { resource: TaskResultResource }) {
153   const task = resource.envelope!.task!;
154   const result = resource.envelope!.result!;
155   const rows = result.metrics.runs;
```

```
front/src/workbench/results/TaskResultViews.tsx
```

```
156   const [focusMetric, setFocusMetric] = useState<EvaluationMetricKey>('acc');
157   const [trendMode, setTrendMode] = useState<EvaluationTrendMode>('absolute');
158   const summary = useMemo(() => deriveEvaluationSummary(result), [result]);
159   const trendOption = useMemo(
160     () => buildEvaluationTrendOption(result, focusMetric, trendMode),
161     [focusMetric, result, trendMode],
162   );
163   const trendRef = useResultChart(trendOption);
164   const focusInsight = summary.metrics.find((metric) => metric.key === focusMetric)
165     ?? summary.metrics[0];
166   const accInsight = summary.metrics.find((metric) => metric.key === 'acc');
167   const representativeRun = result.preview.summary.representativeRun;
168   const overallTone = summary.runCount < 2
169     ? 'neutral'
170     : summary.metrics.some((metric) => metric.stabilityTone === 'warning')
171       ? 'warning'
172       : 'success';
173   const overallLabel = summary.runCount < 2
174     ? '数据不足'
175     : overallTone === 'success'
176       ? '整体波动较低'
177       : '存在波动项';
178
179   function exportMetrics() {
180     downloadTextFile(
181       `task-${task.id}-performance.csv`,
182       buildEvaluationCsv(result),
183       'text/csv;charset=utf-8',
184     );
185   }
186
187   return (
188     <section className="soft-page result-evaluation-page">
189       <ReadyHeader
190         title="性能评估"
191         icon={ClipboardCheck}
192         resource={resource}
193         actions={
194           <div className="result-evaluation-header-actions">
195             <button type="button" className="btn btn-secondary" onClick={resource.refresh}>
196               <RefreshCw size={15} aria-hidden="true" />
197               刷新
198             </button>
199             <button
200               type="button"
201               className="btn btn-primary"
202               disabled={!rows.length}
203               onClick={exportMetrics}>
204               <Download size={15} aria-hidden="true" />
205               导出 CSV
206             </button>
207           </div>
208         }
209       />
210     <MetricStrip result={result} />
211     {rows.length ? (
212       <>
213         <section className="panel result-evaluation-conclusion" aria-label="评估结论">
214           <WorkbenchSectionHeader
```

```
front/src/workbench/results/TaskResultViews.tsx
```

```
217         title="评估结论"
218         meta="基于当前重复实验的汇总结果，未引入外部方法基线"
219         actions=<WorkbenchStatus tone={overallTone}>{overallLabel}</WorkbenchStatus>
220     />
221     <div className="result-evaluation-conclusion-body">
222         <div className="result-evaluation-conclusion-copy">
223             <span className="result-evaluation-conclusion-state">
224                 <CheckCircle2 size={16} aria-hidden="true" />
225                 {summary.runCount} 轮重复实验已完成
226             </span>
227             <strong>四项聚类指标均已完成跨轮次统计</strong>
228             <p>
229                 ACC 均值为 {formatPercent(accInsight?.mean)}, 最高值出现在第 {accInsight?.bestRun?.run ?? '—'} 轮。
230                 {summary.mostStableMetric
231                     ? `${summary.mostStableMetric.label} 的相对波动最低，可作为本次结果稳定性的主要参考。`
232                     : '当前缺少足够轮次，暂时无法判断跨轮次稳定性。'}
233             </p>
234         </div>
235         <dl className="result-evaluation-conclusion-facts">
236             <div>
237                 <dt><Target size={14} aria-hidden="true" />代表轮次</dt>
238                 <dd>第 {representativeRun} 轮</dd>
239                 <small>用于结果预览与后续分析</small>
240             </div>
241             <div>
242                 <dt><Clock3 size={14} aria-hidden="true" />平均耗时</dt>
243                 <dd>{formatNumber(summary.meanRuntime, 2)} s</dd>
244                 <small>累计 {formatNumber(summary.totalRuntime, 2)} s</small>
245             </div>
246             <div>
247                 <dt><Gauge size={14} aria-hidden="true" />波动最低</dt>
248                 <dd>{summary.mostStableMetric?.label ?? '—'}</dd>
249                 <small>变异系数 {summary.mostStableMetric?.variationRatio == null ? '—' :
formatPercent(summary.mostStableMetric.variationRatio)}</small>
250             </div>
251             <div>
252                 <dt><BarChart3 size={14} aria-hidden="true" />波动最高</dt>
253                 <dd>{summary.mostVariableMetric?.label ?? '—'}</dd>
254                 <small>变异系数 {summary.mostVariableMetric?.variationRatio == null ? '—' :
formatPercent(summary.mostVariableMetric.variationRatio)}</small>
255             </div>
256         </dl>
257     </div>
258 </section>
259
260 <div className="result-evaluation-analysis-grid">
261     <section className="panel result-evaluation-trend-panel" aria-label="指标趋势">
262         <WorkbenchSectionHeader
263             title="指标趋势"
264             meta={trendMode === 'relative'
265                 ? ` ${summary.runCount} 个观测点 • 以指标均值为 0 基线显示百分点变化`
266                 : ` ${summary.runCount} 个观测点 • 纵轴按 ${metricLabels[focusMetric]} 区间自动缩放`
267             actions={
268                 <div className="result-evaluation-trend-controls">
269                     <div className="result-evaluation-trend-mode" role="group" aria-label="趋势显示方式">
270                         <button
271                             type="button"
272                             aria-pressed={trendMode === 'absolute'}
273                             onClick={() => setTrendMode('absolute')}
274                         >
275                             原始值
```



```
front/src/workbench/results/TaskResultViews.tsx
```

```
276         </button>
277         <button
278             type="button"
279             aria-pressed={trendMode === 'relative'}
280             onClick={() => setTrendMode('relative')}
281         >
282             相对均值
283         </button>
284     </div>
285     <SelectField
286         value={focusMetric}
287         options={evaluationMetricKeys.map((key) => ({ value: key, label: metricLabels[key] })))}
288         ariaLabel="突出评估指标"
289         className="result-evaluation-metric-select"
290         size="sm"
291         onChange={(value) => setFocusMetric(value as EvaluationMetricKey)}
292     />
293 </div>
294 }
295 />
296 <div className="result-evaluation-trend-summary" aria-label={` ${metricLabels[focusMetric]} 统计摘要`} >
297     <span>均值 <strong>{formatPercent(focusInsight?.mean)}</strong></span>
298     <span>标准差 <strong>{formatPercent(focusInsight?.std)}</strong></span>
299     <span>最佳轮次 <strong>第 {focusInsight?.bestRun?.run ?? '—'} 轮</strong></span>
300 </div>
301 <div
302     className="result-evaluation-trend-chart"
303     ref={trendRef}
304     role="img"
305     aria-label={` ${metricLabels[focusMetric]} ${trendMode === 'relative' ? '相对均值变化趋势' : '跨轮次变化趋势'} `}
306 />
307 </section>
308
309 <section className="panel result-evaluation-stability" aria-label="稳定性诊断">
310     <WorkbenchSectionHeader
311         title="稳定性诊断"
312         meta="变异系数 = 标准差 / 指标均值"
313         actions={<WorkbenchStatus tone={overallTone}>{overallLabel}</WorkbenchStatus>}
314     />
315     <div className="result-evaluation-stability-list">
316         {summary.metrics.map((metric) => (
317             <div className={`result-evaluation-stability-row is-${metric.key}`} key={metric.key}>
318                 <div>
319                     <strong>{metric.label}</strong>
320                     <small>标准差 {formatPercent(metric.std)}</small>
321                 </div>
322                 <span className={`result-evaluation-stability-status is-${metric.stabilityTone}`}>
323                     {metric.stabilityLabel}
324                 </span>
325                 <b>{metric.variationRatio == null ? '—' : formatPercent(metric.variationRatio)}</b>
326             </div>
327         )))
328     </div>
329     <div className="result-evaluation-runtime-band">
330         <Clock3 size={16} aria-hidden="true" />
331         <div>
332             <span>单轮耗时区间</span>
333             <strong>{formatNumber(summary.minimumRuntime, 2)} - {formatNumber(summary.maximumRuntime, 2)}
334             s</strong>
335         </div>
336         <small>平均 {formatNumber(summary.meanRuntime, 2)} s</small>
```

```
front/src/workbench/results/TaskResultViews.tsx
```

```
336         </div>
337         <div className="result-evaluation-method-note">
338             <Info size={15} aria-hidden="true" />
339             <span>稳定性标签仅用于页面内快速筛查，正式结论以标准差、区间和完整轮次明细为准。</span>
340         </div>
341     </section>
342 </div>
343
344 <section className="panel result-evaluation-distribution" aria-label="指标分布">
345     <WorkbenchSectionHeader
346         title="指标分布"
347         meta="逐项核对均值、标准差、跨轮次区间与最高值轮次"
348     />
349     <div className="result-evaluation-distribution-list">
350         {summary.metrics.map((metric) => (
351             <div className={`result-evaluation-distribution-row is-${metric.key}`} key={metric.key}>
352                 <div className="result-evaluation-distribution-name">
353                     <strong>{metric.label}</strong>
354                     <span className={`result-evaluation-stability-status is-${metric.stabilityTone}`}>
355                         {metric.stabilityLabel}
356                     </span>
357                 </div>
358                 <div className="result-evaluation-distribution-values">
359                     <span>均值<strong>{formatPercent(metric.mean)}</strong></span>
360                     <span>标准差<strong>{formatPercent(metric.std)}</strong></span>
361                     <span>区间宽度<strong>{formatPercent(metric.spread)}</strong></span>
362                 </div>
363                 <div
364                     className="result-evaluation-range"
365                     role="img"
366                     aria-label={` ${metric.label} 最小值${formatPercent(metric.min)}, 均值${formatPercent(metric.mean)}, 最大值${formatPercent(metric.max)} `}
367                 >
368                     <span>{formatPercent(metric.min)}</span>
369                     <div className="result-evaluation-range-track" aria-hidden="true">
370                         <i style={{ left: `${metric.meanPosition}%` }} />
371                     </div>
372                     <span>{formatPercent(metric.max)}</span>
373                 </div>
374                 <div className="result-evaluation-best-run">
375                     <Award size={15} aria-hidden="true" />
376                     <span>最高值<strong>第 {metric.bestRun?.run ?? '—'} 轮</strong></span>
377                 </div>
378             </div>
379         )))
380     </div>
381 </section>
382
383 <section className="panel result-evaluation-table-panel" aria-label="重复实验明细">
384     <WorkbenchSectionHeader
385         title="重复实验明细"
386         meta={` ${rows.length} 次运行 • 当前突出 ${metricLabels[focusMetric]} • 随机种子按任务参数派生`}
387         actions={(<WorkbenchStatus tone="info">完整记录</WorkbenchStatus>)}
388     />
389     <div className="result-evaluation-table" role="table" aria-label="重复实验明细表">
390         <div className="result-evaluation-table-row header" role="row">
391             <span role="columnheader">轮次</span>
392             <span role="columnheader">随机种子</span>
393             {evaluationMetricKeys.map((key) => (
394                 <span className={key === focusMetric ? 'is-focused' : ''} role="columnheader" key={key}>
395                     {metricLabels[key]}
```

```
front/src/workbench/results/TaskResultViews.tsx
```

```
396         </span>
397     )))
398     <span role="columnheader">耗时</span>
399     <span role="columnheader">标记</span>
400 </div>
401 {rows.map((row) => {
402     const isRepresentative = row.run === representativeRun;
403     const isBest = row.run === focusInsight?.bestRun?.run;
404     return (
405         <div
406             className={`result-evaluation-table-row${isBest ? ' is-best' : ''}${isRepresentative ? '
is-representative' : ''}`}
407             role="row"
408             key={row.run}
409         >
410             <strong role="cell">第 {row.run} 轮</strong>
411             <span role="cell">{row.seed}</span>
412             {evaluationMetricKeys.map((key) => (
413                 <span className={key === focusMetric ? 'is-focused' : ''} role="cell" key={key}>
414                     {formatPercent(row[key])}
415                 </span>
416             ))}
417             <span role="cell">{formatNumber(row.runtimeSeconds, 2)} s</span>
418             <span className="result-evaluation-row-tags" role="cell">
419                 {isBest ? <b className="is-best">最高 {metricLabels[focusMetric]}</b> : null}
420                 {isRepresentative ? <b className="is-representative">代表轮次</b> : null}
421                 {!isBest && !isRepresentative ? <small>常规轮次</small> : null}
422             </span>
423         </div>
424     );
425     })}
426 </div>
427 </section>
428 </>
429 ) : (
430     <section className="panel result-evaluation-empty" role="status">
431         <Activity size={20} aria-hidden="true" />
432         <div>
433             <strong>当前结果没有重复实验明细</strong>
434             <span>汇总指标仍会保留；完成至少两轮运行后，趋势与稳定性诊断会自动出现。</span>
435         </div>
436     </section>
437 )}
438
439 <section className="panel result-evaluation-context" aria-label="评估口径">
440     <WorkbenchSectionHeader
441         title="评估口径"
442         meta="当前任务的结果复核上下文"
443     />
444     <dl className="result-evaluation-context-grid">
445         <div><dt>数据集</dt><dd>{task.datasetName}</dd></div>
446         <div><dt>算法模式</dt><dd>{task.mode}</dd></div>
447         <div><dt>样本数量</dt><dd>{result.preview.summary.sampleCount}</dd></div>
448         <div><dt>真实类别</dt><dd>{result.preview.summary.classCount}</dd></div>
449         <div><dt>基础聚类</dt><dd>{result.preview.summary.baseClusterCount}</dd></div>
450         <div><dt>重复轮次</dt><dd>{summary.runCount}</dd></div>
451         <div><dt>随机种子起点</dt><dd>{result.preview.summary.randomSeed}</dd></div>
452         <div><dt>代表轮次</dt><dd>第 {representativeRun} 轮</dd></div>
453         <div><dt>任务总耗时</dt><dd>{formatNumber(result.runtimeSeconds, 2)} s</dd></div>
454         <div><dt>实验累计耗时</dt><dd>{formatNumber(summary.totalRuntime, 2)} s</dd></div>
455     </dl>
```

```
front/src/workbench/results/TaskResultViews.tsx
```

```
456     </section>
457   </section>
458   );
459 }
460
461 export function TaskResultViews({ section, resource }: Props) {
462   const logTask = resource.envelope?.task;
463   const isSelectedLogTask = logTask && (!resource.taskId || logTask.id === resource.taskId);
464   if (!resource.error && section === 'logs' && isSelectedLogTask) {
465     return <TaskLogsPage resource={resource} />;
466   }
467
468   if (
469     resource.loading
470     || resource.error
471     || resource.envelope?.state !== 'ready'
472     || !resource.envelope.result
473     || !resource.envelope.task
474   ) {
475     return <ResultState resource={resource} />;
476   }
477
478   // 路由 section 是展示意图，数据来源始终是同一个后端结果资源，避免各页面各自拼装假数据。
479   if (section === 'analysis') return <AnalysisOverviewPage resource={resource} />;
480   if (section === 'ca-matrix') return <CaResultPage resource={resource} />;
481   if (section === 'kernel-config') return <KernelResultPage resource={resource} />;
482   if (section === 'mkl') return <MklResultPage resource={resource} />;
483   if (section === 'evaluation') return <EvaluationPage resource={resource} />;
484   if (section === 'visualization') return <VisualizationResultPage resource={resource} />;
485   if (section === 'results') return <ResultsAnalysisPage resource={resource} />;
486   if (section === 'export') return <ExportResultPage resource={resource} />;
487   if (section === 'reports') return <ReportResultPage resource={resource} />;
488   return <AnalysisOverviewPage resource={resource} />;
489 }
490
```